

Human oversight of agentic systems in practice: Examining the oversight work, challenges, and heuristics of developers using software agents

SHIPI DHANORKAR*, Microsoft, USA

SAMIR PASSI*, Microsoft, USA

MIHAELA VORVOREANU, Microsoft, USA

Autonomous software agents hold promise to increase developer productivity but make mistakes and exhibit novel failure modes, making human oversight central to successful human-agent collaboration. Existing research on agent oversight is largely conceptual; normative frameworks exist, but how users *actually* oversee agents is less known. In this paper, we bridge this gap by providing early empirical anchors for the theoretical discourse on agent oversight. Drawing on interviews with 17 experienced developers, we conduct an exploratory inquiry examining *what* forms of emergent oversight work developers perform, *when*, and *how*. We also document the oversight *challenges* developers face and the *strategies* they have started using to address them. We found at least four forms of emergent oversight work: a priori control, co-planning, real-time monitoring, and post hoc review. We show that oversight work is not only reactive and retrospective, as portrayed in existing research, but also preventative and proactive. We describe situated oversight challenges (e.g., difficulty reviewing agent-generated code) and outline heuristics developers adopt to address such challenges (e.g., using test results as guarantees for code correctness). We conclude with high-level takeaways, future research directions, implications for the human-centered design of software agents and for software engineering practice, and limitations of our research.

CCS Concepts: • **Human-centered computing** → **Empirical studies in HCI**; • **Software and its engineering** → **Software development techniques**; • **Computing methodologies** → **Distributed artificial intelligence**.

Additional Key Words and Phrases: Human Oversight, software agents, AI agents, coding, software engineering

ACM Reference Format:

Shipi Dhanorkar, Samir Passi, and Mihaela Vorvoreanu. 2026. Human oversight of agentic systems in practice: Examining the oversight work, challenges, and heuristics of developers using software agents. In *The 2026 ACM Conference on Fairness, Accountability, and Transparency (FAccT '26)*, June 25–28, 2026, Montreal, QC, Canada. ACM, New York, NY, USA, 28 pages. <https://doi.org/10.1145/3805689.3812402>

1 Introduction

Agentic AI systems are becoming ubiquitous, with applications across tasks such as data analysis, scientific discovery, and web navigation [14, 32, 43, 85, 93, 115]. Unlike conventional AI, agents can autonomously plan, use external tools, and take actions in real-world environments [79, 108, 116, 151]. The impact of agents is perhaps most evident in software engineering, which has seen a proliferation of tools such as Cursor, Claude Code, Cline, ChatGPT Codex, and GitHub Copilot, and in technical advancements such as Open Code Interpreter

*Both authors contributed equally to this research.

Authors' Contact Information: Shipi Dhanorkar, shipidhanorkar@gmail.com, Microsoft, Redmond, USA; Samir Passi, v-sapassi@microsoft.com, Microsoft, Redmond, USA; Mihaela Vorvoreanu, Microsoft, Redmond, USA, mihaela.vorvoreanu@microsoft.com.



This work is licensed under a Creative Commons Attribution 4.0 International License.

FAccT '26, Montreal, QC, Canada

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2596-8/2026/06

<https://doi.org/10.1145/3805689.3812402>

and AlphaEvolve [90, 140, 154]. Specialized *code agents*—systems capable of making repository-level changes [138, 149, 152]—have pushed way beyond simple code generation tools [49, 58] and made their way into developer workflows and organizational practices.

Agentic systems, however, are not without problems. They often fail for several reasons, such as misinterpreting requests, errors during interactions with external tools, inability to identify mistakes, and context loss [7, 17, 55, 82]. Autonomous coding accelerates software development but also paves the way for mistakes that silently propagate into production. Irreversible errors, ranging from localized file deletions [80] to large-scale production incidents, such as the Replit agent’s database erasures [89], pose a significant threat to system reliability. Agents also face cybersecurity risks both as vulnerable systems (e.g., susceptibility to prompt injection attacks, as in Gemini CLI [28]) and as enablers of large-scale autonomous attacks (e.g., credentials and sensitive data leakage, such as in the Claude Code incident [5]). While automated safety methods (e.g., red teaming, agents-as-a-judge) are increasingly used to catch agentic failures [40, 125, 156], they remain limited by the scenarios and incentives they encode [102]. Agents may exhibit biases [6] and behave differently in real deployments than in evaluations [81, 87].

All of this makes *human oversight* of agents a priority. This is also evident in governance frameworks that increasingly mandate human oversight of AI. The EU AI Act explicitly requires that humans retain meaningful control over high-risk AI, and the European Commission’s guidance on trustworthy AI identifies oversight as a core requirement [1]. However, such frameworks assume that oversight is feasible without thoroughly examining or specifying what it looks like in practice. It often falls on users to *do* oversight in their context, ensure agents work properly, and fix agent mistakes and failures [1, 8, 25, 64, 114]. However, users frequently struggle to identify mistakes, even in conventional AI outputs, and exhibit overreliance [91, 95–97]. Agentic systems exacerbate these issues because they introduce new failure modes and require users to assess not only agent outputs but also agent actions [17, 82, 94, 114]. Oversight challenges are particularly acute for developers who must oversee agents across multiple software tasks and projects in production environments under time pressure.

Agent oversight is essential for effective human-agent collaboration and is increasingly seen as an accountability and governance requirement. However, it is remarkably difficult to facilitate or even know *how* to enable effective agent oversight *in practice* [7, 45, 52, 94, 112, 127]. As Section 2 explains, most research on oversight is conceptual or narrowly focused on specific tasks, such as finding mistakes in AI outputs. Even with the pressing need to examine on-the-ground practices, there is little empirical research that holistically examines how users *actually* oversee AI or agentic systems in messy real-world settings, rendering current oversight efforts largely aspirational [69, 94, 133, 136]. We take qualitative, human-centered steps in this direction. We focus on the agent oversight practices of software engineers because they are the professionals most likely to understand the limitations of the technology and the need for oversight. We specifically focus on power users: early adopters in well-resourced organizations and heavy users who can signal emergent practices and challenges and help model oversight best practices. Challenges they encounter are likely to be exacerbated for other user groups. Drawing on research in FAccT and adjacent fields on human oversight and human-AI collaboration and the emergent use and challenges of software agents, we pursue two research questions:

- How do developers oversee software agents in practice?
- What oversight challenges do developers face and how do they address them?

Our goal is to take exploratory steps to highlight the situated work of overseeing software agents. Our participant pool spans multiple organizations but is largely situated within a single tech company. Therefore, we acknowledge that we do not provide fully generalizable findings or an exhaustive taxonomy of oversight work. Our motivation is to provide early empirical anchors to bridge the gaps between current aspirations and the emerging practice of agent oversight. In the rapidly accelerating context of agentic software engineering, our granular and focused insights provide a crucial baseline for understanding the practical aspects of emerging agent oversight work. **We make three key contributions:**

- (1) We provide one of the first *empirical accounts* of the lived reality of developers overseeing software agents. This is a much-needed step towards bridging the theory and practice of agent oversight.
- (2) We present an *exploratory qualitative analysis* of the emergent forms of oversight work developers do in practice, *when*, and *how*, including its *challenges* and the *heuristics* they use to address them.
 - (a) We find developers doing at least four forms of oversight work—a priori control, co-planning, real-time monitoring, and post hoc review—using specific practices to achieve varied goals. We show that oversight is not only reactive (responding during execution) and evaluative (reviewing post-execution), as portrayed in prior research, but also preventative (configuring before prompting) and proactive (co-planning).
 - (b) We highlight oversight challenges developers face (e.g., difficulty reviewing agent-generated code) and document heuristics they adopt to address them (e.g., using test results as guarantees for code correctness). We find that developers opt for efficient, not perfect, oversight, surfacing disconnects between research aspirations for ideal human supervision and the practical realities of using agents.
- (3) We outline future sociotechnical research directions, provide design implications to address developers' immediate oversight challenges, and highlight broader implications for the software engineering practice.

The rest of the paper is organized as follows. Section 2 provides an overview of relevant research to contextualize our research focus, findings, and takeaways. Section 3 describes our data collection and analysis methods, including recruitment and participant information. Section 4 presents the empirical findings. Section 5 discusses high-level takeaways, future research areas, and design and practice implications. We end with limitations of our research and ways to address them.

2 Literature Review

2.1 Human oversight of AI

Although we focus on human oversight of novel software agents, our conceptualization of oversight draws from research on human oversight of conventional AI systems such as predictive algorithms and recommender systems [16, 29, 38, 52, 64, 65, 105]. While a detailed overview of that research is outside the scope of this paper (we do engage with FAccT research on AI oversight in subsection 2.3), two insights are helpful to contextualize this paper's findings and arguments.

The first insight is that reviewing AI outputs is considered key to overseeing any AI system. Researchers debate the specifics of what constitutes—or *should* constitute—oversight, but agree that spotting and correcting mistakes is a crucial part of it [13, 70, 91, 94, 114]. The second insight is that the field of human oversight of AI has significant empirical gaps. Most discourse on oversight of AI, and now agentic AI, either has a *narrow empirical focus* on controlled studies of mostly artificial verification tasks using proxy measures (unsurprising, given the importance attributed to the oversight work of reviewing AI outputs) [e.g.: 12, 44, 46, 63, 72, 107] or is *largely conceptual*, focusing on high-level arguments, frameworks, and theories of oversight [e.g.: 29, 41, 65, 68, 70, 112, 127, 132].

The overall result is a disconnect between the research and practice of human oversight. Empirical research is only beginning to explore how users *actually* oversee AI systems in real-world settings. Without this knowledge, it is difficult to identify *on-the-ground* oversight work, practices, and challenges, let alone build *effective* mechanisms to support oversight in practice. We take steps to bridge this gap by providing exploratory qualitative analyses of the emergent oversight work, challenges, and heuristics of developers using software agents.

2.2 Agents in software development: Applications and challenges

The 2021 breakthrough demonstrating the ability of large language models (LLMs) to analyze and generate software code [21] sparked widespread interest, driving significant technical advancements [116] including agentic frameworks such as OpenHands CodeAct [137]. Coding agents have since been applied to a wide range of software engineering tasks [54, 76], such as debugging and patching [9, 153], code translation [37, 144], code

refactoring [53, 130, 147], and generation tests for bug reproduction or proof of vulnerability [2, 24, 83, 88]. Alongside technical advancements, developer productivity studies have demonstrated the positive impact of software agents on task time [92, 100]. Sarkar’s [106] natural experiment analyses measured another productivity metric, repository merge rates, and found that organizations using Cursor’s agent achieved 26% more merges than those that did not use it. Coding agents continue to show impressive performance on benchmark evaluations, most prominently on SWE-bench and its related leaderboard [59].

However, software agents also exhibit critical limitations. Studies have revealed severe flaws in LLM-generated code, including hallucinations [56, 124, 129], security vulnerabilities [99], reasoning-action misalignment [10], and novel attacks originating from the MCP infrastructure [123]. Agents also falter on execution errors, such as database integrity violations and operating system errors that persist despite evaluations [23]. The explosion of benchmarks such as SWE-Bench, RedCode, BigCodeBench, and CodeLM-Sec [47, 48, 59, 157] exposes coding agents’ rapid evolution alongside their deepening flaws such as memory pitfalls [3, 74, 150], test-passing but functionally broken patches [139, 155], and failure to adhere to complex instructions with embedded constraints [148]. However, a gap remains: controlled evaluations do not reflect the high-stakes complexity of real-world software lifecycles. This underscores the necessity of human supervision to mitigate risks and ensure real-world reliability in agent-driven software development.

Although formal research on developers’ oversight practices is still lacking, we see glimpses of the need for and importance of human supervision in emerging human-centered research on agent-led software development. A recent study made visible the benefits of developer revisions to agentic pull requests handling critical bugs, highlighting the need for human involvement to ensure the correctness of agent-generated code [141]. Recent comparative studies between conventional AI coding tools and modern software agents document significant workflow shifts, noting that as agents take over task execution, humans tend to concentrate more on monitoring and debugging agent actions [4, 22]. Kumar et al. [67] explicitly connect successful agentic outcomes with active, iterative developer involvement. Through field observations and a qualitative survey, Huang et al. [57] suggest that experienced developers rarely vibe code, relying on iteratively steering and editing agent outputs, because they regard one-shot generations as unreliable. Another research shows novice developers struggle to leverage agents effectively because of usability issues [120], highlighting tensions between industry visions of autonomous agents and developers’ on-the-ground experiences and challenges.

2.3 The emerging focus on human oversight of agentic systems in FAccT scholarship

Human oversight of AI has also emerged as an important FAccT research topic, largely under the umbrella of human-AI collaboration, with researchers drawing connections between oversight and longstanding FAccT topics of *accountability* [33, 135], *auditing* [15], *fairness* [50], *harm* [19, 142], and *responsibility* [103, 135]. A common thread across this research is the link between oversight and transparency. Meaningful oversight requires more than just placing humans in loops. Users need “epistemic access” to systems to adequately understand them [127]. Transparency is key to enabling such access, and FAccT researchers continue to examine transparency’s role and limits for human-AI collaboration (including the work of reviewing AI outputs) [27, 50, 60, 75, 104, 113]. Shang et al. [113] demonstrate how counterfactual explanations can improve human-AI decision making. Kim et al. [61] show how linguistic expressions of uncertainty in AI outputs help reduce overreliance. Schuff et al. [110], however, advise caution, arguing that users can misinterpret explanations because of biases and superficial factors. Information provided by AI systems can be difficult to understand for some users and mislead others [133], exacerbating automation bias [127] and overreliance [73, 109]. We draw on these insights when analyzing the information needs and practices of developers overseeing software agents. We contribute findings on the *transparency challenges* and *reviewing practices* of developers through a “human-centered analysis [to further] define the design space” [69, p.1375] of human oversight and human-AI collaboration.

Much like wider research on human oversight, FAccT research has also mostly focused on conventional AI. Researchers have only recently turned their attention to GenAI and agents, with early insights hinting that such systems exacerbate oversight needs and challenges. Venkit et al. [86] focus on the additional burden of verifying GenAI outputs. Wang et al. [136] show that GenAI systems often lack adequate support for output evaluation. Agents introduce further risks; e.g., “reward hacking” makes agents produce right outputs for wrong reasons [19], making it necessary for users to review both the outputs *and* working of agentic systems [19, 66, 135].

Addressing agentic oversight requirements and challenges, Chan et al. [18] posit, requires designing new forms of “visibility” into the complex composition and working of agentic systems. Concurrently, such new designs must also account for the trade-offs additional oversight aids introduce between system understanding, user workload, and human cognition [30, 84, 94, 104, 114]. As researchers in FAccT and adjacent fields have recently pointed out, there is a pressing need for more empirical, human-centered research to examine how users navigate such tradeoffs when working with agentic systems in practice, likely “compensat[ing] for unmet” needs [133] through oft-invisible situated actions and intuitions [11, 20, 39, 69, 94, 133, 136].

Our exploratory analysis of agent oversight in practice is a step toward this goal. We contribute to FAccT research on agent oversight and human-agent collaboration by analyzing and documenting the different oversight work developers perform, the challenges and tradeoffs they face when doing different oversight work, and the heuristics developers use to navigate tradeoffs and work around such challenges.

3 Research Methodology

In the subsections below, we provide details on our participation criteria, recruitment process, participants, data collection, and analysis strategy.

3.1 Participants: Participation criteria, recruitment process, and organizational context

We followed a criterion-sampling strategy based on experience using software agents for professional work. In this study, we define a *software agent* as one that autonomously plans and acts to accomplish user goals by using tools and interacting with the environment.¹ Our recruitment materials included examples, such as Claude Code and Cline, to clarify what we meant. We did not consider an AI that just auto-completes code as an agent.

We first reached out to a few professional contacts, and employed snowball sampling to identify additional participants. Participants filled out a screener to determine participation eligibility. We assessed our sampling criteria through a combination of participants’ self-reported *agent experience* and *agent usage for professional work*. We only selected those who were *somewhat experienced* with agents and used them at least *weekly* for professional work. Sixteen of 17 participants used agents several times a week for professional work (12 daily). Nine participants reported being ‘experienced’ with agents, four ‘somewhat experienced,’ and four ‘very experienced.’² See Appendix A for further participant details.

Ours is also an intensity sample [98], concentrating on experienced developers who likely understand agentic AI better than people in other professions. Thirteen participants had seven or more years of programming experience, two had over four years, and two had more than a year, spanning domains such as ad experimentation, financial technology, research, and storage systems. Challenges seasoned developers experience are likely worse for practitioners with less accurate mental models of agentic AI. We also specifically focused on power users in

¹Given this definition, our work is informed by existing research but has a distinct goal. For example, we extend Chen et al.’s [22] research identifying oversight as a new form of work with the emerging mechanics of that work and Wang et al.’s [136] research on trust in modern AI systems with a focus on autonomous agents.

²Since agentic AI is still an emerging tech in its nascency, we deliberately chose to not ask for participants’ agent experience in months/years and instead opted for open-ended language in the screener (e.g., ‘somewhat’ and ‘very’ experienced). To further ensure we get relevant participants, we used the interview’s opening questions as a secondary screener to assess participants’ conceptual understanding and usage of agents. We excluded one participant based on secondary screening.

well-resourced organizations, as early adopters and heavy users are well-positioned to signal emergent practices and help model best practices.

Twelve of 17 participants work at the same large-scale tech organization as the authors. This occurred unintentionally due to participant recruitment challenges. We wanted to interview experienced software agent users, but a large part of that user base consists of developers working in the tech industry. NDAs, proprietary nature of agent work, and the competitive agent landscape made developers from other firms hesitant to participate in our study. Software agent users at our organization felt more comfortable participating and sharing details. Organizational culture impacts practices but norms can vary across teams in the same company. The 12 participants were on distinct teams across domains with no overlapping projects; they shared similarities (e.g., security requirements) but also exhibited differences (e.g., using different agents for tasks). The 5 participants from other organizations shared similar experiences.

3.2 Data collection and analysis

We used a semi-structured approach for interviews. Rather than present a preconceived definition of oversight, we let participants explain what agent oversight meant to them and how they conducted it. We first asked developers to describe what they used agents for (see Appendix B for agent usage details). We then dived into how developers worked with agents, evaluated agents' work (e.g., spotting mistakes in agent outputs), and the difficulties they faced in doing so. The structure of our findings (i.e., four forms of oversight work) is shaped by our analytical lens (i.e., explicitly asking users what they do before, during, and after agent execution). We deliberately used temporal anchoring to ground participant responses in end-to-end agent workflows. The structure of our findings is thus best seen as a useful organizing device and not an exact representation. Note, however, that participants did organize their responses temporally even without prompting when differentiating between the oversight work of *co-planning* and *real-time monitoring*. See Appendix C for details on interview questions.

Interviews were conducted 1:1 by the first author via video conferencing software, video-recorded, and transcribed using Marvin [78]. Participants completed a consent form prior to the interview. Interviews were scheduled for one hour, lasting 60 minutes on average, and conducted between July-August 2025. Participants received a US \$40 gift card or equivalent for participation. The research study was approved by our organization's institutional review board.

We followed a qualitative inductive analysis process guided by our research questions and in concert with existing literature [131]. Analysis proceeded in three rounds. In round one, the first author conducted open coding, creating a list of developer actions, practices, and challenges through in-vivo codes directly using participant quotes (e.g., “set deny lists” and “co-plan with agents”). In round two, the first author captured common themes across open codes, creating 13 axial codes (e.g., *co-planning* and *review strategies*). In round three, the first and second authors conducted selective coding, developing the final two conceptual categories that form our findings: different forms of oversight work (subsection 4.1) and the heuristics used to work around oversight challenges (subsection 4.2). The first and second authors regularly met during the second and third rounds to compare and resolve ambiguities.³

³For example, initially the authors differed on whether to keep *a priori control* (subsection 4.1.1) and *co-planning* (subsection 4.1.2) as distinct categories or merge them into one “control” category. On revisiting the coded data, a functional boundary between the two emerged: configuring pre-established controls vs. actively steering agents. Consequently, these were maintained as distinct axial codes. In another instance, the authors initially differed on the nature of heuristics — one saw them as risky shortcuts while the other saw them as coping mechanisms. This was resolved by revisiting coded data to analyze participants' motivation in employing heuristics, revealing them as imperfect-yet-necessary shortcuts to do *good enough* post hoc review.

4 Findings

Our data analysis process yielded two key themes—one related to emergent forms and challenges of oversight (subsection 4.1) and another to the situated practical strategies developers use to perform oversight work despite its challenging nature (subsection 4.2). Appendix B describes participants' agent usage scenarios to help contextualize the emergent agent oversight work we describe in this section.

4.1 Four forms of human oversight work

Our exploratory analysis revealed four forms of emergent oversight work that developers perform when using software agents: a priori control, co-planning work, real-time monitoring, and post hoc review (see Appendix D).

4.1.1 A priori control. A priori control is a form of oversight work that involves giving instructions to agents to direct and limit their workings *before* delegating tasks to them. The goal of a priori control is to define clear boundaries for agents and ensure they are set up properly to minimize failure.⁴ Developers exercise a priori control using built-in agent settings and custom instructions. A priori control includes configuring agent autonomy, stopping agents from using specific code libraries, and supplying global context to agents on project scope.

“There’s one essential thing in this puzzle: your instruction. [...] A specific file in the repo that contains base instructions to understand the repo for the agent. That’s the most important asset [...] that you’re constantly maintaining, improving, and iterating.” (P15)

A priori control is especially important because of the inherent unpredictability of agents—*“sometimes [the agent] follows your instructions, sometimes it tries to go on a tangent and do its own stuff”* (P11). The specifics of what developers do for a priori control depend on the agentic failures they imagine or have previously encountered.

All participants acknowledged the need to constrain agents but differed in the nature and extent of the a priori control work they performed. Some *“spen[t] a lot of time setting up or configuring”* agents (P17), specifying detailed *“dependenc[ies] and external contracts”* (P14). P03 explained, *“I will try to set it up as much as possible, [e.g.] I update the deny list where I don’t want the agent to ever delete something.”* Other developers simply relied on default agent controls:

“I have seen some videos where you can redefine some actions and flows, but I have really not gone too much into that. [...] I just use it. ‘Agentic mode’ is the default mode. [...] I just use that. I have not messed around with the configurations too much, to be honest.” (P13)

Challenges in a priori control. We observed two challenges to effective a priori control. The first is that despite the presence of a priori control mechanisms, developers perceive having little control over an agent’s working. As P04 said, they *“don’t think [they] have”* much control over an agent’s *actual* operation—a sentiment echoed by several participants. One reason for this perception is that participants primarily used prompts, and not custom instructions, to control agents, even though experienced developers argued that using custom instructions *“drives directly the quality of what the agent produce[s]”* (P15). Using black-boxed agents made matters worse:

“I didn’t set up the exact agents that we use to assign tasks. We had to request from another team to have it integrated. I don’t think we choose which specific model. It’s probably their team that set that up, and we just sort of get the end-user impact of having the agent working.” (P02)

The second challenge is that developers must make informed choices while working with limited information about agents. For example, P16 said they were *“not even sure what reasoning [agents] are using.”* Beyond agent operation, developers also have limited information about agents’ data policies—e.g., what *“happen[s] to all this*

⁴Despite a priori control, developers *“never know”* for certain how agents will behave, leading developers to resort to working in *“very secure environments”* such as a *“sandbox”* (P16) to experiment and understand the agent’s working.

data that is going [in]to these models" (P01), *"especially if it is [...] customer-related data"* (P03) being transmitted to third-party platforms.

4.1.2 Co-planning work. Co-planning is a form of oversight work that involves preliminary goal-setting activities that agents and developers jointly perform to establish common ground. The goal of co-planning is to ensure agents identify key actionable requirements for successful task completion, minimize misalignment with user intent, and facilitate downstream oversight. Examples include iterative prompting, drafting plans with agents, and seeding partial solutions to agents. Co-planning is important because letting agents operate completely hands-off often results in their failure to accurately fill decision gaps independently.

"I learned as a hard lesson: do not completely go hands-off. [...] LLMs are short-term memory thing. [...] No problem can be solved only in one way, right? It could be solved in thousands of ways, and it will start flipping between those thousand ways, creating a mess because decisions are not consistent across the project. Sometimes it will make a decision—an important technical decision—in one part, and hours later, it will just forget about it and start again and make some other decision. It will be a hodgepodge of a project. It will be very difficult to make it work." (P05)

Participants described two key co-planning practices. The first is drafting plans with agents. Through this collaborative work, agents and developers align on how best to accomplish goals by, for instance, building a shared understanding of the solution's *"data flow, structure, key components, [and] how they're linked"* (P07). The desire to minimize review and repair downstream motivates developers to invest time and effort up front during co-planning: *"While we are iterating software with these AI coding agents, it's better to have a design up front compared to making code changes later"* (P14). This is especially important when there are many possible solutions and developers want agents to implement specific ones: *"if there are multiple approaches, I tell what is the correct approach, and I ask it to follow the [company's] coding practices"* (P08). Developers often seed partial implementations during co-planning to further guide agents:

"Sometimes, in the planning mode, it's not getting to the point that I want to. Then I write the code snippet. I bite the bullet. I finally write the code and say, 'Okay, this is the code snippet.' I don't write full code. I'm still a lazy developer. [...] I write the crux of it." (P11)

The second co-planning practice participants described is task decomposition. Developers *"deconstruct"* tasks into small *"chunks"* where each *"has minimal side effects and can be testable independently"* (P17). Smaller tasks help *"contain"* (P05) and *"narrow down"* (P14) code changes to fewer files and scoped functions, making review and change easier. As P05 explained, with smaller tasks:

"the margin of error is very low...this function is expecting this outcome in this format and providing this parameter. I know what to do with this. If that gap is bigger and it has to take a decision and it doesn't have context of the rest of the application, it would more likely build something that is not well integrated with the rest of the application." (P05)

Coherent mental models facilitate co-planning. When decomposing tasks, knowing when minimum details can suffice (e.g., when there are *"well-defined, state-of-the-art solutions"* (P06) agents can easily find) and when agents require detailed instructions (e.g., where there are *"crazy risk conditions based on logs, metrics, [...]and] you need to provide all this context to the AI"* (P16)) is key to breaking down tasks effectively to steer agents towards success. Similarly, when drafting plans with agents, a *"sense of familiarity [for] things that [the agent] gets wrong vs. things it gets right almost all the time"* (P13) helps make informed decisions about guiding agents.

Overall, participants called out the iterative nature of co-planning work that includes, for instance, not only providing details, but also reviewing and refining plans: *"once you have received a response, if it was like 10% better than the previous response, you have to go on to more specifics"* (P04). Co-planning involves negotiating with agents: *"Is this right?," "why do you think we need two classes?," and "why do you lay out folders this way?"* (P14).

Participants described co-planning as an emerging form of *spec-driven development* (P05, P15) that ensures agents remain “grounded” (P05, P07):

“First you write specs, then you have your agent that explains [... them,] does deep research on your issue [...], runs, and then you ask further questions to that agent until you’ve built sufficient context. Then you run a last agent that does a [final] summary and creates a prompt that you use to launch the software agent.” (P15)

Challenges in co-planning. We observed two key challenges to effective co-planning. First, it is difficult to identify the appropriate level of specificity at which to instruct agents. What is “obvious” to humans is often invisible to agents (P01). Experienced developers do not just solve coding problems but also consider implicit requirements such as security, performance, and maintainability (P04, P06). However, unlike humans, who can infer missing information and adjust on the fly, agents make brittle decisions without detailed instructions. Under-specified plans cause agents to misinterpret goals and overlook constraints (P03, P05, P11). However, developers “don’t always know in advance” (P11) how much hand-holding agents need or at what point hand-holding work overshadows the benefits of using agents altogether (P02).

“If you provide a high level plan, you get high level code. [...] You tell me that I’m going to go from X to Y, but then I’m going to stop at T for a break. You implement a solution like X, Y, T. You say stop at T, but the coding solution shouldn’t be that. The coding solution should introduce a new parameter: how much time are you going to stop at T?” (P04)

The second challenge is that using natural language for prompting makes it difficult to appropriately articulate and specify goals to agents, widening alignment gaps between developer intent and agent working. For simple things, developers often find themselves “over explaining” to agents: “I will say things like, ‘do this in Python,’ even though to me it’s obvious that it’s Python” (P01). In complex situations, developers frequently second guess whether their intent is clear enough for agents to interpret or whether the provided examples are sufficient for agents to identify the correct solution.

“Making the prompt coherent is like writing a story. Constructing that story in my head in a coherent way is not easy. I can have bits and pieces of that story. Code should do this, do that. [But] having a flow of story that if this happened, do this, that’s not an easy task for a human. [...] Articulating stuff in the time you write the prompt, like 15 seconds, [...] that’s my challenge, I think a universal challenge.” (P06)

4.1.3 Real-time monitoring. Real-time monitoring is a form of oversight work that involves observing agents as they perform tasks and intervening when needed. The goal of real-time monitoring is to ensure agents remain on track, are not stuck in loops or failure modes, and follow predefined plans. Examples of real-time monitoring work include observing action logs, reading reasoning traces, and pausing and stopping agents mid-execution.

Participants revealed that they rarely performed real-time monitoring, with most never “proactively look[ing] at the [agent’s] thought process” (P12). The few times participants monitored action logs or reasoning traces, they did so perfunctorily: “eyeball[ing] [them] to see that [the agent] did the right thing based on what it said” (P13). Especially for longer running and complex tasks, relying solely on quick scans is problematic, causing developers to miss important details and allow subtle agent errors to slip through.

We hypothesize that a key reason for our participants not engaging in real-time monitoring lies in how they assigned tasks to agents. As described earlier, developers decomposed large tasks into smaller sub tasks during co-planning to improve the intelligibility of intermediate agent outputs and facilitate post hoc testing. Delegating small tasks to agents meant that agents worked quickly on mostly simple tasks, making it relatively easy for participants to make do either with quick glances during execution or with skipping real-time monitoring altogether and relying only on reviewing final agent outputs.

Challenges in real-time monitoring. Since participants rarely monitored agents in real time, they did not describe specific challenges with it. However, our analysis revealed three observations about the *general* difficulty of real-time monitoring as described by our participants.

The first observation is on the consequential disconnect between what an agent says it is doing and what it actually does. For example, developers regularly deal with agents' poor confidence calibration in reasoning traces: *"10% of the time it is wrong [about why it did something]. It is very confident, so you don't get to know."* (P10). Therefore, while artifacts such as reasoning traces and Chain-of-thought (CoT) rationales provide monitoring capabilities, they remain inherently unreliable.

The second observation is on the use of ad hoc cues to identify possible issues during real-time monitoring. Participants relied on indirect cues such as abnormally long agent run time or conversation turns, and not vigilant monitoring, to spot issues such as context drift during execution (P03, P06, P17).

"One of the biggest things I observed [...] is that if you go a certain distance with the agent, then its hallucination starts growing. Like it has too much context over a period. If you spend 30 minutes or one hour, it has a big context. It starts doing random things. Things I didn't even ask." (P03)

The third observation is on the difficulty of fixing agent mistakes during execution. When developers spot issues mid-execution, they work to address them and bring the agent back on track by, for instance, pausing the agent and manually updating its context or prompting it to fix mistakes before moving on. However, participants mentioned (P02, P17) that such approaches were inefficient and unreliable:

"I find that if you try to redirect [the agent] and give it the right context, it doesn't get back on track. I have to sort of stop it and then update my prompt and restart all over again." (P02)

4.1.4 Post hoc review. Post hoc review is a form of oversight work that involves reviewing and correcting agent outputs after execution. It was the most discussed oversight work in our study. The goal of post hoc review is to ensure agent outputs are correct and are achieved using appropriate approaches. Examples of post hoc review work include verifying agent outputs, making adjustments when agents falter, and re-prompting agents to fix issues. Post hoc review is important because agents can not only make legitimate mistakes (e.g., incomplete code that does not handle all edge cases), but also hallucinate (e.g., mentioning *"classes that don't exist in codebases"* (P05)). Developers conduct post hoc review through different practices such as reviewing code outputs using diff tools, using LLMs-as-a-judge, and layered testing (e.g., agent-generated test suites or manual testing of code functionality).

"After the code gen, I get into the code diff [...] and start going through each file. It's how I review others' code: get to each file and, as with code reviews, you first see overall things, then you figure out some key features or code areas that it has modified. Go to those first, see if it's good or not, then start updating a few smaller things." (P11)

The extent of post hoc review work depends on the type of tasks developers delegate to agents. For instance, consider the difference between incremental and ground-up software development. Incremental software development is entrenched in legacy codebases and intertwined with interconnecting modules. *"There [is] a lot of legacy code, a lot of modules, so the complexity becomes more because it's not generic."* (P04). Using agents to integrate new features in existing codebases makes rigorous post hoc review work even more crucial, at times *"more stringently than if it was a person [who wrote the code]"* (P02).

"I read every single line. I approve every single line. [...] For serious software engineering, especially where you're part of a broader system, it's extremely problematic if you have code where you're not exactly sure what every single line does. You show up in a meeting, there are people who need to make integration decisions that rely on you knowing exactly how this code works." (P17)

Unlike incremental software, ground-up software has little dependency on existing codebases or prior modules. Examples include proof-of-concept and research prototypes. Despite valuing post hoc review for functional validation, participants admitted granting their agents “*unrestricted*” (P16) autonomy and maintaining reduced scrutiny in such cases.

“I do a lot of prototyping. [If] I just want something that will let me demo but I don’t really care about the design. In that case, I’m okay with letting the agent just run wild.” (P01)

Challenges in post-hoc review. We observed developers facing two key challenges that inhibit effective post hoc review. The first challenge is that using agents makes developers cognitively distant from the code they must review. This increases the “*cost or effort [developers must] put into verification, testing, [and] benchmarking*” (P17). As agents code, developers drift further from the practice of writing code. “*A computer [is] writing the code, but humans need to read it*” (P14). However, as P01 put it, “*because [the code] is not created by me, my understanding of it is very surface level.*” “*It’s not your code you’re reviewing, it’s somebody else’s. I can review my code much faster. It’s like similar stuff with your handwriting. It is much faster for you to read [your own] than somebody else’s handwriting*” (P11). The growing distance between developers and agent-generated code makes it harder to troubleshoot even simple issues in agent outputs, especially because the sheer “volume of [agent-generated] code is so high” (P13).

“A couple of times I wanted to make some changes. [...] A very small change, like font size or something. [...] I ended up spending quite a bit of time understanding the code, and it was a lot harder to understand because I didn’t write it.” (P01)

The second challenge is that developers must re-review agent-generated code with every iteration (P11, P14). When developers identify an issue with agent output and ask the agent to fix it, they feel the need to review the whole code again because the agent might have made changes, and at times unnecessary changes, in multiple places. “*Each time you say, ‘okay, generate again or fix it again,’ you have to review it again*” (P11).

4.2 Four heuristics developers use for post hoc review

The previous subsection showed different oversight work and highlighted several challenges. Our analysis revealed that faced with the complex reality of overseeing software agents, developers have begun adopting heuristics that prioritize efficiency over perfection to work *around* the challenges. Below we describe four such heuristics participants used to accomplish the oversight work of *post hoc review*. We focus only on post hoc review because it was the most discussed oversight work among participants, and thus heuristics used for it were most prominently visible during analysis.

Heuristic #1: An agent’s plan is a faithful proxy for its actual working. Participants heavily relied on the plan during post hoc review, treating it as a faithful representation of the agent’s actual working and equating the quality of the agent’s output with the seeming quality of the agent’s plan (P05, P07, P14, P15, P16). This heuristic assumes that agents adhere to plans during execution and that an agent’s working is congruent with plan contents, enabling developers to substitute the large effort of code review with the easier task of plan verification. We see this most saliently at play in P16’s argument against the very need to understand an agent’s “thought process” during post hoc review:

“When I ask [an agent] to do something, I don’t need to understand the whole thought process. It’s there, which I love, right? It’s doing something so that I get a feel of its thinking. But I don’t read through every single thing it does. The only thing that matters is the end: what did you do? [...] I just look at the plan, and then if I don’t like something, I mention it. I don’t care what it does in between. Whether it’s thinking, searching, looking, I don’t go through it line by line. That’s what everyone does. Maybe there’s a different way of doing it, but that’s just how I do it.” (P16)

The fact that developers spend a lot of up front effort co-drafting plans with agents is further evidence of this proxy-based approach. Developers are willing to even change their workflows, adopting spec-driven development approaches assuming that agents will adhere to plans:

“I have to watch, understand, critique every outcome or steps it is taking, which is not scalable if I have to move fast. [...] What I pivoted to is creating a system, an approach that will help me do that instead of me doing that manually. This spec-driven development is part of that system: hey, let’s solidify PRD before getting to tech spec; solidify tech spec before getting into UX; then create a full implementation plan before we even write a single line of code; then create task sections and tasks that LLM can blindly follow and get the right output.” (P05)

Heuristic #2. Passing test results guarantee the correctness of agent-generated code. Participants often outsourced verification to the test suite. When agent-generated code passes testing, it lends confidence in the code’s correctness—the agent “*did the job*” (P03). In contrast, when tests fail, developers treat them as a way to identify which parts of agent outputs are problematic and must be evaluated during post hoc review. As P11 mentioned, going through test results instead of inspecting every line of agent-generated code is highly efficient:

“[When] the rubber meets the road, files get updated and I review the code, run the test cases. All the time, what I do is, after [the agent] writes code, I always [...] and we have agreed on test cases... I tell [the agent] to also ensure that you run tests and stuff and show me test results. Because it’s easier to review test results and see it’s bad, not working, and then iterate on it [...] than for me to go through the code.” (P11)

As several participants mentioned, the viability of this heuristic—i.e., reviewing test results in place of actual code—depends on the quality of tests themselves (notably P08, P11, P14). Indeed, as P08 surmised, it is possible to fully eradicate the need to review *any* agent-generated code if developers can guarantee good test cases:

“We can ask AI or a human to write test cases, but humans need to review and be 100% responsible for test cases. We need to make sure that the spec 100% captures what the PM or the designer really wants. If that can be guaranteed, we can treat the source folder as a complete black box. If AI is able to pass all test cases and, of course, build, then we give it a go and we don’t even need to take a look at the source folder anymore. When there’s new changes, new requirements, just update test cases and let AI regenerate everything. There’s no attachment that we have to the code.” (P08)

Heuristic #3. Eyeballing agent-related information, including but not limited to agent outputs, can reliably signal issues. A main way in which participants managed information overload during post hoc review is through spot-checking by eyeballing agent-related information such as agent outputs, agent rationales, and change summaries. The assumption underlying this heuristic seems to be that eyeballing suffices as an incomplete-yet-efficient information processing mechanism during review. As P07 described: “*I have this way of doing spot checks or high-level checks to make sure [the agent] is not missing some important files in the analysis.*” P13 made a similar observation:

“Before I push any code, I look at all method signatures and give it an eyeball: ‘Okay, is this method needed?’ [If] I see a method that I don’t think makes sense over here, I’ll just ask, ‘Is it actually needed? Should we remove it?’ [...] The agent also] gives me the reasoning and then does some work, then gives me another piece of reasoning, does some work. I can see the work it is doing is directly related to the reasoning that it is giving me. At the end, it summarizes: ‘these are the changes I made; what is the logic behind it?’ I can easily eyeball it and see, okay, it did the right thing based on what it said.” (P13)

We observed developers not just eyeballing existing information but also asking agents to generate *new* information such as visual diagrams to guide and facilitate post hoc review.

“I will ask the agent to go through the entire code path and build a data flow diagram for me. Go through all function calls and follow data flow of the entire pull request and generate a Mermaid-compatible diagram. This will give me the pseudo code so I can copy and put it in the Mermaid visualization tool to view the structure and data flow. This is really helpful for big PRs to understand whether all the key components are in place and connected in the right way!” (P07)

Heuristic #4. It is reasonable to trust agents when dealing with new information or unfamiliar contexts.

Participants showed signs of automation bias, trusting agents’ approaches and outputs when working in new domains or dealing with new information. P14, for instance, explicitly called this out: *“sometimes there are very technical details, like using certain libraries a certain way and I have no knowledge about the library. That is the hard part. I usually will have to trust the model. I never used it, so yes you are right!”* This epistemic deference suggests trust by necessity; lacking expertise and knowledge, developers move away from explicit verification to implicit reliance. Such deference may occur occasionally during a software project, but can also extend to the entire project itself:

“I shipped a feature in Go. I did not know Go a month ago. I’m not a fan of Go. Yeah, it worked. [...] I’m not a judge of Go, but it works. I reviewed every line and they looked legit.” (P15)

We also observed an interesting variant of ‘social proof’ in the context of this heuristic. When working in uncharted waters, agreement between two *different* agents on the same task increases users’ trust in agents:

“I usually use the same prompt on both [agents] just to have some more safety. If both are showing similar things, I have higher confidence it’s giving me a grounded and comprehensive report.” (P07)

5 Discussion and Implications

Our findings provide an early look into the emerging work, challenges, and heuristics of developers overseeing software agents. Our qualitative account of the lived reality of overseeing software agents is a contribution in and of itself, providing much-needed empirical anchors to start bridging the theory and practice of agent oversight in at least one specific domain. Below we describe two key takeaways, future research directions, and implications for the human-centered design of software agents and for the software engineering practice.

5.1 Oversight entails not only reviewing and monitoring, but also controlling and steering

Our findings reveal that oversight work is likely not only reactive and evaluative, but also preventative and proactive. *Post hoc review* is the most visible form of oversight work in our study and in research [13, 70, 91, 94]. Additionally, although our participants did little *real-time monitoring*, it is seen as a growing challenge for agent oversight [18, 50, 94, 135]. However, participants described that oversight involves more than just reviewing agent outputs and monitoring agent actions. Developers also *control* and *steer* agents. Oversight work begins—as *a priori control*—even before prompting. For instance, developers anticipate failures, preemptively configuring agents to prevent them from making specific mistakes. Oversight work is also underway—as *co-planning*—before agents start executing tasks. For instance, developers revise plans with agents and seed solutions to them, pushing agents in desirable directions away from pitfalls.

Some of the oversight work we found fits traditional notions of oversight (e.g., reviewing and monitoring), while some (e.g., preemptive configuration) appear less like oversight and more like “good” software engineering practices. However, participants described doing such anticipatory “pre” work at times with *explicit* oversight intentions (e.g., preventing known mistakes or failures from occurring altogether). This is not to say that our findings warrant redefining oversight to now include software engineering practices. Instead, surfacing the

oversight aspirations underpinning control and steering work, we highlight that oversight work is emerging not just in expected spaces, but also elsewhere in prompts and settings.

We show different oversight work occurring *throughout* human-agent interaction, expanding research scholarship on *what* users do as humans-in/on/over-loops (e.g., we show that users anticipate, and not just react) and *where* oversight loops begin/end (e.g., we show that they begin long before prompting) [25, 29, 46, 135, 158]. Our insights into *post hoc review* and *real-time monitoring* extend existing research on transparency-driven approaches to human-AI and human-agent collaboration [50, 104, 127]. Our insights into *a priori control* and *co-planning* surface new research sites such as the role of “anticipation” [26, 35, 126], the affordances and limits of natural language-based human-agent communication [7, 77, 94, 119], and the “managerial” work users must do when using agents (and its impact on craftsmanship, learning, and professional identity) [31, 132, 143].

5.2 Oversight functions not through exhaustive awareness, but through practical heuristics

As explained in subsection 2.1, existing research largely aspires for “effective” oversight by facilitating continuous, “meaningful” human control over AI and agents [16, 29, 105, 114, 117]. However, our findings show that such aspirations often prove unscalable and are decoupled from the practical realities of users running software agents. We found that during *post hoc review* participants opted for good-enough supervision driven by “bounded rationality” [121] rather than exhaustive awareness of agentic outputs and workings. Faced with complex agentic systems, developers adopt emergent heuristics that prioritize functional efficiency over idealized perfection. Plans become proxies for agentic working, test results stand in for code correctness, and skimming becomes the main information processing mechanism. These are not lazy shortcuts but necessary adaptations users do to “practically accomplish” [42, 111] oversight by working *around* constraints. Agentic systems will always be “too fast, too complex, and too much for users,” [94, p. 18] requiring them to inevitably use heuristics that “satisfice” rather than “optimize” oversight [51, 62, 101, 122]. We outline emergent heuristics to start reconciling theoretical work on human oversight with its situated reality [18, 64, 127], inviting normative discussions around which heuristics are worth embracing and which introduce risks that must be mitigated [45, 118, 134].

5.3 Implications for the human-centered design of software agents

While not all oversight challenges described in subsection 4.1 are immediately solvable (e.g., most need further research or technological advancements), it is possible to address certain challenges and facilitate agent oversight.

- (1) Developers struggle during *a priori control* because of insufficient information on agents and under-reliance on custom instructions. Agent builders can address this by providing configuration guides that explain how users can control specific elements of agent behavior, by surfacing constraints and defaults early to help developers anticipate downstream agent behavior, and by providing visible, low-effort ways for users to leverage custom instructions.
- (2) Developers struggle during *co-planning* because of difficulties formulating user goals as agent prompts and decomposing tasks optimally. Agent builders can address this by offering prompting help to users through context-aware suggestions and by providing suggested sub-tasks recommendations combined with their resource costs, reasoning depth, and execution previews.
- (3) Developers struggle during *real-time monitoring* to fix agent mistakes, using ad hoc cues to identify them. Agent builders can address this by designing interfaces that help users better understand and guide agent actions [146] (e.g., via reasoning panels to inspect incorrect learnings and inline controls to fix them) and by surfacing contextual signals to aid issue identification (e.g., via visualizing agent memory and time data).
- (4) Developers struggle during *post hoc review* because of the large cognitive distance between developers and agent-generated code. Agent builders can address this by reducing friction in verifying agent outputs (e.g., augment code diff tools with intent, reasoning, and proposed tests) and by providing ways to help

developers reverse engineer agent logic (e.g., interweave rationale and reasoning traces, and visualize impact of changes on agent output).

5.4 Implications for the software engineering practice: Individuals and organizations

The traditional ‘craftsman’ model of software engineering is giving way to a new role: the ‘developer-manager’, wherein hands-on coding is increasingly becoming secondary to the work of oversight. This shift carries significant implications. *First, for the individual developer*, the core competency is shifting from syntax mastery to architectural design, critique, and review. This requires developers to have the ability to understand system limitations and detect anomalies (mandated by EU AI act’s Article 14(4)(a) [1]) and its success hinges on balancing managerial judgment and the technical intuition required for final vetting. How developers take on new agent oversight work and develop new oversight skills while meaningfully preserving their coding skills will shape their professional identity [36]. *Second, for engineering organizations*, human oversight is evolving from a best practice into a strict legal mandate [1], likely creating tensions with the efficiency-driven heuristics we saw in our study. To remain compliant, organizations must ensure developers, especially those working with high risk AI systems, possess the situational awareness and epistemic access necessary to resist automation bias in fast-paced workflows.

6 Limitations and Future Work

This research takes important first steps and provides valuable contributions on agent oversight in practice but is not without limitations. *First*, while we tried recruiting participants from diverse professional settings, most participants worked in well-resourced technology firms (e.g., generous token limits, rich tool integrations, and early access to new features). Monetary and computational costs mattered little in their oversight decisions. Future work must examine oversight challenges and implications in resource-constrained settings. *Second*, 12 of our 17 participants worked at the same technology company. Although their experiences were echoed by the 5 participants from other firms, one company’s over-representation likely introduced some bias in our findings. Future research on oversight in diverse organizational contexts can complement this research with a more holistic view of agent oversight work. *Third*, our participants delegated relatively small tasks to agents, rendering, for example, real-time monitoring optional and post hoc review easier. Future research must analyze agent oversight work in the context of complex, longer-running tasks, while also examining how agent builders conceptualize and facilitate these more demanding forms of oversight work and requirements.

7 Conclusion

Our study provides an exploratory analyses of how software developers oversee coding agents in practice. Developers perform diverse oversight work using specific practices to achieve different goals. We identify pain points developers encounter when supervising agents and the emergent heuristics they adopt to navigate the intrinsic uncertainty of working with agents. Our findings reveal the nuanced ways users have begun exercising oversight, beginning as early as during configuration and planning, and continuing through ongoing verification. *Doing* oversight is less of a single act and more of a coordinated set of ad-hoc evolving practices. By starting to make these activities visible, we contribute early empirical anchors for understanding, researching, and improving agent oversight. Our work also reflects on broader implications for tooling, individuals, organizations and future research to study, design, and support oversight in practice to enable effective collaboration and complementarity between humans and agents.

Acknowledgments

We thank our research participants for sharing their experiences with us. We express our gratitude to Jenna Butler for her insights during the study’s design phase, to Dean Carignan, Ben Zorn and Siddhi Tadpatrikar for

their assistance with participant recruitment. We also thank our colleagues Amy K. Heger and Kathleen Walker for help with this research. Last but not the least, we thank the anonymous reviewers for their constructive feedback.

Generative AI Usage

The authors used generative AI tools for transcribing interviews, proofreading, and at times to improve the fluency of sentences. Generative AI was not used for data analysis or generating technical content.

References

- [1] E.U. Artificial Intelligence Act. 2024. Article 14: Human Oversight | EU Artificial Intelligence Act. <https://artificialintelligenceact.eu/article/14/> [Online; accessed 2025-08-01].
- [2] Toufique Ahmed, Jatin Ganhotra, Rangeet Pan, Avraham Shinnar, Saurabh Sinha, and Martin Hirzel. 2025. Otter: Generating Tests from Issues to Validate SWE Patches. In *Forty-second International Conference on Machine Learning*. <https://openreview.net/forum?id=b0jYs6JOZu>
- [3] Ali Al-Kaswan, Maliheh Izadi, and Arie van Deursen. 2024. Traces of Memorisation in Large Language Models for Code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal) (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 78, 12 pages. doi:10.1145/3597503.3639133
- [4] Anthropic. 2025. Anthropic Economic Index report: Uneven geographic and enterprise AI adoption — anthropic.com. <https://www.anthropic.com/research/anthropic-economic-index-september-2025-report>. [Online; accessed 12-01-2026].
- [5] Anthropic. 2025. Disrupting the first reported AI-orchestrated cyber espionage campaign. <https://assets.anthropic.com/m/ec212e6566a0d47/original/Disrupting-the-first-reported-AI-orchestrated-cyber-espionage-campaign.pdf> [Online; accessed 2026-01-09].
- [6] Gagan Bansal, Wenyue Hua, Zezhou Huang, Adam Fourney, Amanda Swearngin, Will Epperson, Tyler Payne, Jake M. Hofman, Brendan Lucier, Chinmay Singh, Markus Mobius, Akshay Nambi, Archana Yadav, Kevin Gao, David M. Rothschild, Aleksandrs Slivkins, Daniel G. Goldstein, Hussein Mozannar, Nicole Immorlica, Maya Murad, Matthew Vogel, Subbarao Kambhampati, Eric Horvitz, and Saleema Amershi. 2025. Magentic Marketplace: An Open-Source Environment for Studying Agentic Markets. arXiv:2510.25779 [cs.MA] <https://arxiv.org/abs/2510.25779>
- [7] Gagan Bansal, Jennifer Wortman Vaughan, Saleema Amershi, Eric Horvitz, Adam Fourney, Hussein Mozannar, Victor Dibia, and Daniel S. Weld. 2024. Challenges in Human-Agent Communication. arXiv:2412.10380 [cs.HC] <https://arxiv.org/abs/2412.10380>
- [8] Joseph R Biden. 2023. Executive order on the safe, secure, and trustworthy development and use of artificial intelligence. *Presidential Actions* (2023).
- [9] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. 2025. RepairAgent: An Autonomous, LLM-Based Agent for Program Repair. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (Ottawa, Ontario, Canada) (ICSE '25)*. IEEE Press, 2188–2200. doi:10.1109/ICSE55347.2025.00157
- [10] Islem Bouzenia and Michael Pradel. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. arXiv:2506.18824 [cs.SE] <https://arxiv.org/abs/2506.18824>
- [11] Michelle Brachman, Siya Kunde, Sarah Miller, Ana Fucs, Samantha Dempsey, Jamie Jabbour, and Werner Geyer. 2025. Building Appropriate Mental Models: What Users Know and Want to Know about an Agentic AI Chatbot. In *Proceedings of the 30th International Conference on Intelligent User Interfaces (IUI '25)*. Association for Computing Machinery, New York, NY, USA, 247–264. doi:10.1145/3708359.3712071
- [12] Zana Bućinca, Maja Barbara Malaya, and Krzysztof Z. Gajos. 2021. To Trust or to Think: Cognitive Forcing Functions Can Reduce Overreliance on AI in AI-assisted Decision-making. *Proc. ACM Hum.-Comput. Interact.* 5, CSCW1, Article 188 (April 2021), 21 pages. doi:10.1145/3449287
- [13] Stefan Buijsman and Herman Veluwenkamp. 2023. Spotting When Algorithms Are Wrong. *Minds and Machines* 33, 4 (01 Dec 2023), 541–562. doi:10.1007/s11023-022-09591-0
- [14] Ruisheng Cao, Fangyu Lei, Haoyuan Wu, Jixuan Chen, Yeqiao Fu, Hongcheng Gao, Xinzhuang Xiong, Hanchong Zhang, Wenjing Hu, Yuchen Mao, et al. 2024. Spider2-v: How far are multimodal agents from automating data science and engineering workflows? *Advances in Neural Information Processing Systems* 37 (2024), 107703–107744.
- [15] Stephen Casper, Carson Ezell, Charlotte Siegmann, Noam Kolt, Taylor Lynn Curtis, Benjamin Bucknall, Andreas Haupt, Kevin Wei, Jérémy Scheurer, Marius Hobbhahn, Lee Sharkey, Satyapriya Krishna, Marvin Von Hagen, Silas Alberti, Alan Chan, Qinyi Sun, Michael Gerovitch, David Bau, Max Tegmark, David Krueger, and Dylan Hadfield-Menell. 2024. Black-Box Access is Insufficient for Rigorous AI Audits. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (Rio de Janeiro, Brazil) (FAccT '24)*. Association for Computing Machinery, New York, NY, USA, 2254–2272. doi:10.1145/3630106.3659037

- [16] Luciano Cavalcante Siebert, Maria Luce Lupetti, Evgeni Aizenberg, Niek Beckers, Arkady Zgonnikov, Herman Veluwenkamp, David Abbink, Elisa Giaccardi, Geert-Jan Houben, Catholijn M. Jonker, Jeroen van den Hoven, Deborah Forster, and Reginald L. Lagendijk. 2023. Meaningful human control: actionable properties for AI system development. *AI and Ethics* 3, 1 (01 Feb 2023), 241–255. doi:10.1007/s43681-022-00167-3
- [17] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why Do Multi-Agent LLM Systems Fail? arXiv:2503.13657 [cs.AI]
- [18] Alan Chan, Carson Ezell, Max Kaufmann, Kevin Wei, Lewis Hammond, Herbie Bradley, Emma Bluemke, Nitarshan Rajkumar, David Krueger, Noam Kolt, Lennart Heim, and Markus Anderljung. 2024. Visibility into AI Agents. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency* (Rio de Janeiro, Brazil) (FAccT '24). Association for Computing Machinery, New York, NY, USA, 958–973. doi:10.1145/3630106.3658948
- [19] Alan Chan, Rebecca Salganik, Alva Markelius, Chris Pang, Nitarshan Rajkumar, Dmitrii Krasheninnikov, Lauro Langosco, Zhonghao He, Yawen Duan, Micah Carroll, Michelle Lin, Alex Mayhew, Katherine Collins, Maryam Molamohammadi, John Burden, Wanru Zhao, Shalaleh Rismani, Konstantinos Voudouris, Umang Bhatt, Adrian Weller, David Krueger, and Tegan Maharaj. 2023. Harms from Increasingly Agentic Algorithmic Systems. In *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency* (Chicago, IL, USA) (FAccT '23). Association for Computing Machinery, New York, NY, USA, 651–666. doi:10.1145/3593013.3594033
- [20] Chacha Chen, Shi Feng, Amit Sharma, and Chenhao Tan. 2023. Machine Explanations and Human Understanding. In *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency* (Chicago, IL, USA) (FAccT '23). Association for Computing Machinery, New York, NY, USA, 1. doi:10.1145/3593013.3593970
- [21] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgén Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] <https://arxiv.org/abs/2107.03374>
- [22] Valerie Chen, Ameet Talwalkar, Robert Brennan, and Graham Neubig. 2025. Code with Me or for Me? How Increasing AI Automation Transforms Developer Workflows. arXiv:2507.08149 [cs.SE] <https://arxiv.org/abs/2507.08149>
- [23] Zhi Chen, Wei Ma, and Lingxiao Jiang. 2025. Beyond Final Code: A Process-Oriented Error Analysis of Software Development Agents in Real-World GitHub Scenarios. arXiv:2503.12374 [cs.SE] <https://arxiv.org/abs/2503.12374>
- [24] Runxiang Cheng, Michele Tufano, Jürgen Cito, José Cambronero, Pat Rondon, Renyao Wei, Aaron Sun, and Satish Chandra. 2025. Agentic Bug Reproduction for Effective Automated Program Repair at Google. *CoRR* abs/2502.01821 (2025). arXiv:2502.01821 doi:10.48550/ARXIV.2502.01821
- [25] Ria Cheruvu. 2025. *Human-in/on-the-Loop Design for Human Controllability*. Springer Nature Singapore, Singapore, 1–47. doi:10.1007/978-981-97-8440-0_75-1
- [26] Adele E. Clarke. 2016. Anticipation Work: Abduction, Simplification, Hope. In *Boundary Objects and Beyond: Working with Leigh Star*. The MIT Press. doi:10.7551/mitpress/10113.003.0007
- [27] Eric Corbett and Remi Denton. 2023. Interrogating the T in FAccT. In *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency* (Chicago, IL, USA) (FAccT '23). Association for Computing Machinery, New York, NY, USA, 1624–1634. doi:10.1145/3593013.3594104
- [28] Sam Cox. 2025. Code Execution Through Deception: Gemini AI CLI Hijack. <https://tracebit.com/blog/code-exec-deception-gemini-ai-cli-hijack> [Online; accessed 2026-01-09].
- [29] Lorrie Faith Cranor. 2008. A framework for reasoning about the human in the loop. In *Proceedings of the 1st Conference on Usability, Psychology, and Security* (San Francisco, California) (UPSEC'08). USENIX Association, USA, Article 1, 15 pages.
- [30] Alejandro Cuadron, Dacheng Li, Wenjie Ma, Xingyao Wang, Yichuan Wang, Siyuan Zhuang, Shu Liu, Luis Gaspar Schroeder, Tian Xia, Huanzhi Mao, Nicholas Thumiger, Aditya Desai, Ion Stoica, Ana Klimovic, Graham Neubig, and Joseph E. Gonzalez. 2025. The Danger of Overthinking: Examining the Reasoning-Action Dilemma in Agentic Tasks. arXiv:2502.08235 [cs.AI] <https://arxiv.org/abs/2502.08235>
- [31] Nigel Daly. 2025. Managerial AI Skill Stacking: A New Professional skillset for the AI-Driven Workplace. *Available at SSRN* 5277762 (2025), 11 pages. doi:10.2139/ssrn.5277762
- [32] Mike D'Arcy, Tom Hope, Larry Birnbaum, and Doug Downey. 2024. MARG: Multi-Agent Review Generation for Scientific Papers. arXiv:2401.04259 [cs.CL] <https://arxiv.org/abs/2401.04259>
- [33] Joseph Donia. 2022. Normative Logics of Algorithmic Accountability. In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency* (Seoul, Republic of Korea) (FAccT '22). Association for Computing Machinery, New York, NY, USA, 598. doi:10.1145/3531146.3533123

- [34] Ekwa Duala-Ekoko and Martin P. Robillard. 2012. Asking and answering questions about unfamiliar APIs: An exploratory study. In *2012 34th International Conference on Software Engineering (ICSE)*. 266–276. doi:10.1109/ICSE.2012.6227187
- [35] Upol Ehsan, Q. Vera Liao, Samir Passi, Mark O. Riedl, and III Daumé, Hal. 2024. Seamless XAI: Operationalizing Seamless Design in Explainable AI. *Proc. ACM Hum.-Comput. Interact.* 8, CSCW1, Article 119 (April 2024), 29 pages. doi:10.1145/3637396
- [36] Upol Ehsan, Samir Passi, Koustuv Saha, Todd McNutt, Mark O Riedl, and Sara Alcorn. 2026. From Future of Work to Future of Workers: Addressing Asymptomatic AI Harms to Foster Dignified Human-AI Interaction. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26)*. Association for Computing Machinery, New York, NY, USA, Article 595, 21 pages. doi:10.1145/3772318.3791081
- [37] Hasan Ferit Eniser, Hanliang Zhang, Cristina David, Meng Wang, Maria Christakis, Brandon Paulsen, Joey Dodds, and Daniel Kroening. 2024. Towards Translating Real-World Code with LLMs: A Study of Translating to Rust. *CoRR* (2024).
- [38] Lena Enqvist. 2023. 'Human oversight' in the EU artificial intelligence act: what, when and by whom? *Law, Innovation and Technology* 15, 2 (2023), 508–535. doi:10.1080/17579961.2023.2245683
- [39] Will Epperson, Gagan Bansal, Victor C Dibia, Adam Fourney, Jack Gerrits, Erkang (Eric) Zhu, and Saleema Amershi. 2025. Interactive Debugging and Steering of Multi-Agent AI Systems. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 156, 15 pages. doi:10.1145/3706598.3713581
- [40] Michael Feffer, Anusha Sinha, Wesley H. Deng, Zachary C. Lipton, and Hoda Heidari. 2025. *Red-Teaming for Generative AI: Silver Bullet or Security Theater?* AAAI Press, 421–437.
- [41] Joel Frenette. 2023. Ensuring human oversight in high-performance AI systems: A framework for control and accountability. *World Journal of Advanced Research and Reviews* 20, 2 (2023), 1507–1516.
- [42] Harold Garfinkel and Harvey Sacks. 1987. On formal structures of practical actions. In *Ethnomethodological studies of work*, Harold Garfinkel (Ed.). Routledge, 165–198. <https://doi.org/10.4324/9780203996867>
- [43] Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Anil Palepu, Petar Sirkovic, Artiom Myaskovsky, Felix Weissenberger, Keran Rong, Ryutaro Tanno, Khaled Saab, Dan Popovici, Jacob Blum, Fan Zhang, Katherine Chou, Avinatan Hassidim, Burak Gokturk, Amin Vahdat, Pushmeet Kohli, Yossi Matias, Andrew Carroll, Kavita Kulkarni, Nenad Tomasev, Yuan Guan, Vikram Dhillon, Eeshit Dhaval Vaishnav, Byron Lee, Tiago R D Costa, José R Penadés, Gary Peltz, Yunhan Xu, Annalisa Pawlosky, Alan Karthikesalingam, and Vivek Natarajan. 2025. Towards an AI co-scientist. arXiv:2502.18864 [cs.AI] <https://arxiv.org/abs/2502.18864>
- [44] Navita Goyal, Eleftheria Briakou, Amanda Liu, Connor Baumler, Claire Bonial, Jeffrey Micher, Clare R. Voss, Marine Carpuat, and Hal Daumé III. 2023. What Else Do I Need to Know? The Effect of Background Information on Users' Reliance on QA Systems. arXiv:2305.14331 [cs.CL] <https://arxiv.org/abs/2305.14331>
- [45] Ben Green. 2022. The flaws of policies requiring human oversight of government algorithms. *Computer Law & Security Review* 45 (2022), 105681. doi:10.1016/j.clsr.2022.105681
- [46] Ben Green and Yiling Chen. 2019. The Principles and Limits of Algorithm-in-the-Loop Decision Making. *Proc. ACM Hum.-Comput. Interact.* 3, CSCW, Article 50 (Nov. 2019), 24 pages. doi:10.1145/3359152
- [47] Chengquan Guo, Xun Liu, Chulin Xie, Andy Zhou, Yi Zeng, Zinan Lin, Dawn Song, and Bo Li. 2024. RedCode: Risky Code Execution and Generation Benchmark for Code Agents. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 106190–106236. doi:10.52202/079017-3369
- [48] Hossein Hajipour, Keno Hassler, Thorsten Holz, Lea Schonherr, and Mario Fritz. 2024. CodeLMsec Benchmark: Systematically Evaluating and Finding Security Vulnerabilities in Black-Box Code Language Models. In *2024 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. IEEE Computer Society, Los Alamitos, CA, USA, 684–709. doi:10.1109/SaTML59370.2024.00040
- [49] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring Coding Challenge Competence With APPS. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*. <https://openreview.net/forum?id=sD93GOzH3i5>
- [50] Thomas Henzinger, Mahyar Karimi, Konstantin Kueffner, and Kaushik Mallik. 2023. Runtime Monitoring of Dynamic Fairness Properties. In *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency* (Chicago, IL, USA) (FAccT '23). Association for Computing Machinery, New York, NY, USA, 604–614. doi:10.1145/3593013.3594028
- [51] Erik Hollnagel. 2009. *The ETTO Principle: Efficiency-Thoroughness Trade-Off: Why Things That Go Right Sometimes Go Wrong*. CRC press.
- [52] Andreas Holzinger, Kurt Zatloukal, and Heimo Müller. 2025. Is human oversight to AI systems still possible? *New Biotechnology* 85 (2025), 59–62. doi:10.1016/j.nbt.2024.12.003
- [53] Kosei Horikawa, Hao Li, Yutaro Kashiwa, Bram Adams, Hajimu Iida, and Ahmed E. Hassan. 2025. Agentic Refactoring: An Empirical Study of AI Coding Agents. arXiv:2511.04824 [cs.SE](<http://cs.se/>) <https://arxiv.org/abs/2511.04824>
- [54] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (2024), 1–79.

- [55] Kung-Hsiang Huang, Akshara Prabhakar, Sidharth Dhawan, Yixin Mao, Huan Wang, Silvio Savarese, Caiming Xiong, Philippe Laban, and Chien-Sheng Wu. 2025. CRMArena: Understanding the Capacity of LLM Agents to Perform Professional CRM Tasks in Realistic Environments. *arXiv:2411.02305 [cs.CL]* <https://arxiv.org/abs/2411.02305>
- [56] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. 2025. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Trans. Inf. Syst.* 43, 2, Article 42 (Jan. 2025), 55 pages. doi:10.1145/3703155
- [57] Ruanqianqian Huang, Avery Reyna, Sorin Lerner, Haijun Xia, and Brian Hempel. 2025. Professional Software Developers Don't Vibe, They Control: AI Agent Use for Coding in 2025. *arXiv:2512.14012 [cs.SE]* <https://arxiv.org/abs/2512.14012>
- [58] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, and Luke Zettlemoyer. 2018. Mapping Language to Code in Programmatic Context. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii (Eds.). Association for Computational Linguistics, Brussels, Belgium, 1643–1652. doi:10.18653/v1/D18-1192
- [59] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-world Github Issues?. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VTF8yNQM66>
- [60] Harmanpreet Kaur, Eytan Adar, Eric Gilbert, and Cliff Lampe. 2022. Sensible AI: Re-imagining Interpretability and Explainability using Sensemaking Theory. In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency* (Seoul, Republic of Korea) (FAccT '22). Association for Computing Machinery, New York, NY, USA, 702–714. doi:10.1145/3531146.3533135
- [61] Sunnie S. Y. Kim, Q. Vera Liao, Mihaela Vorvoreanu, Stephanie Ballard, and Jennifer Wortman Vaughan. 2024. "I'm Not Sure, But...": Examining the Impact of Large Language Models' Uncertainty Expression on User Reliance and Trust. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency* (Rio de Janeiro, Brazil) (FAccT '24). Association for Computing Machinery, New York, NY, USA, 822–835. doi:10.1145/3630106.3658941
- [62] Rob Kling. 1980. Social Analyses of Computing: Theoretical Perspectives in Recent Empirical Research. *ACM Comput. Surv.* 12, 1 (March 1980), 61–110. doi:10.1145/356802.356806
- [63] Artur Klingbeil, Cassandra Grütznier, and Philipp Schreck. 2024. Trust and reliance on AI — An experimental study on the extent and costs of overreliance on AI. *Computers in Human Behavior* 160 (2024), 108352. doi:10.1016/j.chb.2024.108352
- [64] Riikka KOULU. 2020. Human control over automation: EU policy and AI ethics. *European journal of legal studies* 12 (2020), 9–46.
- [65] Riikka Koulou. 2020. Proceduralizing control and discretion: Human oversight in artificial intelligence policy. *Maastricht Journal of European and Comparative Law* 27, 6 (2020), 720–735. *arXiv:https://doi.org/10.1177/1023263X20978649* doi:10.1177/1023263X20978649
- [66] Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, Zac Kenton, Jan Leike, and Shane Legg. 2020. Specification gaming: the flip side of AI ingenuity. <https://deepmind.google/discover/blog/specification-gaming-the-flip-side-of-ai-ingenuity/> [Online; accessed 2025-09-01].
- [67] Aayush Kumar, Yasharth Bajpai, Sumit Gulwani, Gustavo Soares, and Emerson Murphy-Hill. 2025. Why AI Agents Still Need You: Findings from Developer-Agent Collaborations in the Wild. *arXiv:2506.12347 [cs.SE]* <https://arxiv.org/abs/2506.12347>
- [68] Kyriakos Kyriakou and Janna Otterbacher. 2023. In humans, we trust. *Discover Artificial Intelligence* 3, 1 (12 Dec 2023), 44. doi:10.1007/s44163-023-00092-2
- [69] Vivian Lai, Chacha Chen, Alison Smith-Renner, Q. Vera Liao, and Chenhao Tan. 2023. Towards a Science of Human-AI Decision Making: An Overview of Design Space in Empirical Human-Subject Studies. In *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency* (Chicago, IL, USA) (FAccT '23). Association for Computing Machinery, New York, NY, USA, 1369–1385. doi:10.1145/3593013.3594087
- [70] Markus Langer, Kevin Baum, and Nadine Schlicker. 2024. Effective Human Oversight of AI-Based Systems: A Signal Detection Perspective on the Detection of Inaccurate and Unfair Outputs. *Minds and Machines* 35, 1 (05 Nov 2024), 1. doi:10.1007/s11023-024-09701-0
- [71] Thomas D. LaToza, Gina Venolia, and Robert DeLine. 2006. Maintaining mental models: a study of developer work habits. In *Proceedings of the 28th International Conference on Software Engineering* (Shanghai, China) (ICSE '06). Association for Computing Machinery, New York, NY, USA, 492–501. doi:10.1145/1134285.1134355
- [72] Min Hun Lee and Chong Jun Chew. 2023. Understanding the Effect of Counterfactual Explanations on Trust and Reliance on AI for Human-AI Collaborative Clinical Decision Making. *Proc. ACM Hum.-Comput. Interact.* 7, CSCW2, Article 369 (Oct. 2023), 22 pages. doi:10.1145/3610218
- [73] Yoonjoo Lee, Kihoon Son, Tae Soo Kim, Jisu Kim, John Joon Young Chung, Eytan Adar, and Juho Kim. 2024. One vs. Many: Comprehending Accurate Information from Multiple Erroneous and Inconsistent AI Generations. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency* (Rio de Janeiro, Brazil) (FAccT '24). Association for Computing Machinery, New York, NY, USA, 2518–2531. doi:10.1145/3630106.3662681
- [74] Shanchao Liang, Spandan Garg, and Roshanak Zilouchian Moghaddam. 2025. The SWE-Bench Illusion: When State-of-the-Art LLMs Remember Instead of Reason. *arXiv:2506.12286 [cs.AI]*(<http://cs.ai/>) <https://arxiv.org/abs/2506.12286>
- [75] Gabriel Lima, Nina Grgić-Hlača, Jin Keun Jeong, and Meeyoung Cha. 2022. The Conflict Between Explainable and Accountable Decision-Making Algorithms. In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency* (Seoul, Republic

- of Korea) (FAccT '22). Association for Computing Machinery, New York, NY, USA, 2103–2113. doi:10.1145/3531146.3534628
- [76] Michael R Lyu, Baishakhi Ray, Abhik Roychoudhury, Shin Hwei Tan, and Patanamon Thongtanunam. 2025. Automatic programming: Large language models and beyond. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–33.
- [77] Ramesh Manuvinakurike, Emanuel Moss, Elizabeth Anne Watkins, Saurav Sahay, Giuseppe Raffa, and Lama Nachman. 2025. Thoughts without Thinking: Reconsidering the Explanatory Value of Chain-of-Thought Reasoning in LLMs through Agentic Pipelines. arXiv:2505.00875 [cs.AI] <https://arxiv.org/abs/2505.00875>
- [78] Marvin. [n. d.]. Marvin: The AI-Native Customer Feedback Repository — heymarvin.com. <https://heymarvin.com/>. [Online; accessed 12-01-2026].
- [79] Tula Masterman, Sandi Besen, Mason Sawtell, and Alex Chao. 2024. The Landscape of Emerging AI Agent Architectures for Reasoning, Planning, and Tool Calling: A Survey. arXiv:2404.11584 [cs.AI] <https://arxiv.org/abs/2404.11584>
- [80] Cecily Mauran. 2025. <https://mashable.com/article/google-gemini-deletes-users-code>
- [81] METR. [n. d.]. *Details about METR's evaluation of OpenAI GPT-5 | METR's Autonomy Evaluation Resources*. <https://evaluations.metr.org/gpt-5-report/?ref=bounded-regret.ghost.io#we-do-find-evidence-of-significant-situational-awareness-though-it-is-not-robust-and-often-gets-things-wrong>
- [82] Microsoft. 2024. *Taxonomy of Failure Mode in Agentic AI Systems*. Technical Report. Microsoft. <https://cdn-dynmedia-1.microsoft.com/is/content/microsoftcorp/microsoft/final/en-us/microsoft-brand/documents/Taxonomy-of-Failure-Mode-in-Agentic-AI-Systems-Whitepaper.pdf>
- [83] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin T. Vechev. 2024. Code Agents are State of the Art Software Testers. *CoRR* abs/2406.12952 (2024). <https://doi.org/10.48550/arXiv.2406.12952>
- [84] Suchismita Naik, Amanda Snellinger, Austin L. Toombs, Scott Saponas, and Amanda K Hall. 2025. Exploring Early Adopters' Use of AI Driven Multi-Agent Systems to Inform Human-Agent Interaction Design: Insights from Industry Practice. In *Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems (CHI EA '25)*. Association for Computing Machinery, New York, NY, USA, Article 677, 8 pages. doi:10.1145/3706599.3706693
- [85] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. 2021. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332* (2021).
- [86] Pranav Narayanan Venkit, Philippe Laban, Yilun Zhou, Yixin Mao, and Chien-Sheng Wu. 2025. Search Engines in the AI Era: A Qualitative Understanding to the False Promise of Factual and Verifiable Source-Cited Responses in LLM-based Search. In *Proceedings of the 2025 ACM Conference on Fairness, Accountability, and Transparency (FAccT '25)*. Association for Computing Machinery, New York, NY, USA, 1325–1340. doi:10.1145/3715275.3732089
- [87] Joe Needham, Giles Edkins, Govind Pimpale, Henning Bartsch, and Marius Hobbhahn. 2025. Large Language Models Often Know When They Are Being Evaluated. arXiv:2505.23836 [cs.CL] <https://arxiv.org/abs/2505.23836>
- [88] Vikram Nitin, Baishakhi Ray, and Roshanak Zilouchian Moghaddam. 2025. Faultline: Automated proof-of-vulnerability generation using llm agents. *arXiv preprint arXiv:2507.15241* (2025).
- [89] Beatrice Nolan. 2025. AI-powered coding tool wiped out a software company's database in 'catastrophic failure' | Fortune. <https://fortune.com/2025/07/23/ai-coding-tool-replit-wiped-database-called-it-a-catastrophic-failure/> [Online; accessed 2026-01-09].
- [90] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131 [cs.AI] <https://arxiv.org/abs/2506.13131>
- [91] Joshua W. Ohde, Lauren M. Rost, and Joshua D. Overgaard. 2025. The Burden of Reviewing LLM-Generated Content. *NEJM AI* 2, 2 (2025), A1p2400979. doi:10.1056/A1p2400979
- [92] Elise Paradis, Kate Grey, Quinn Madison, Daye Nam, Andrew Macvean, Vahid Meimand, Nan Zhang, Ben Ferrari-Church, and Satish Chandra. 2025. How Much Does AI Impact Development Speed? an Enterprise-Based Randomized Controlled Trial. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 618–629. doi:10.1109/ICSE-SEIP66354.2025.00060
- [93] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface, Software and Technology*. 1–22.
- [94] Samir Passi. 2025. Agentic AI has a Human Oversight Problem. Available at SSRN 5529058 (2025), 30 pages. doi:10.2139/ssrn.5529058
- [95] Samir Passi, Shipi Dhanorkar, and Mihaela Vorvoreanu. 2024. *Appropriate reliance on Generative AI: Research synthesis*. Technical Report MSR-TR-2024-7. Microsoft. <https://www.microsoft.com/en-us/research/publication/appropriate-reliance-on-generative-ai-research-synthesis/>
- [96] Samir Passi, Shipi Dhanorkar, and Mihaela Vorvoreanu. 2025. *Addressing Overreliance on AI*. Springer Nature Singapore, Singapore, 1–34. doi:10.1007/978-981-97-8440-0_98-1

- [97] Samir Passi and Mihaela Vorvoreanu. 2022. *Overreliance on AI: Literature Review*. Technical Report MSR-TR-2022-12. Microsoft. <https://www.microsoft.com/en-us/research/publication/overreliance-on-ai-literature-review/>
- [98] M.Q. Patton. 2014. *Qualitative Research & Evaluation Methods: Integrating Theory and Practice*. SAGE Publications. <https://books.google.com/books?id=ovAkBQAAQBAJ>
- [99] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. 2025. Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. *Commun. ACM* 68, 2 (Jan. 2025), 96–105. doi:10.1145/3610721
- [100] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirel. 2023. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv:2302.06590 [[cs.SE](http://cs.se/)] <https://arxiv.org/abs/2302.06590>
- [101] Peter Pirolli and Stuart Card. 1999. Information foraging. *Psychological Review* 106, 4 (1999), 643–675. doi:10.1037/0033-295X.106.4.643
- [102] Inioluwa Deborah Raji, Emily Denton, Emily M. Bender, Alex Hanna, and Amandalynne Paullada. 2021. AI and the Everything in the Whole Wide World Benchmark. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*. <https://openreview.net/forum?id=j6NxpQbREA1>
- [103] Abdul Razzaq, Jim Buckley, Qin Lai, Tingting Yu, and Goetz Botterweck. 2024. A Systematic Literature Review on the Influence of Enhanced Developer Experience on Developers' Productivity: Factors, Practices, and Recommendations. *ACM Comput. Surv.* 57, 1, Article 13 (Oct. 2024), 46 pages. doi:10.1145/3687299
- [104] Lindsay Sanneman, Mycal Tucker, and Julie A. Shah. 2024. An Information Bottleneck Characterization of the Understanding-Workload Tradeoff in Human-Centered Explainable AI. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (Rio de Janeiro, Brazil) (FAccT '24)*. Association for Computing Machinery, New York, NY, USA, 2175–2198. doi:10.1145/3630106.3659032
- [105] Filippo Santoni de Sio and Jeroen van den Hoven. 2018. Meaningful Human Control over Autonomous Systems: A Philosophical Account. *Frontiers in Robotics and AI* Volume 5 - 2018 (2018). doi:10.3389/frobt.2018.00015
- [106] Suproteem K Sarkar. 2025. AI Agents, Productivity, and Higher-Order Thinking: Early Evidence From Software Development. *Available at SSRN* 5713646 (11 2025). doi:10.2139/ssrn.5713646
- [107] Max Schemmer, Niklas Kuehl, Carina Benz, Andrea Bartos, and Gerhard Satzger. 2023. Appropriate Reliance on AI Advice: Conceptualization and the Effect of Explanations. In *Proceedings of the 28th International Conference on Intelligent User Interfaces (Sydney, NSW, Australia) (IUI '23)*. Association for Computing Machinery, New York, NY, USA, 410–422. doi:10.1145/3581641.3584066
- [108] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems* 36 (2023), 68539–68551.
- [109] Vera Schmitt, Luis-Felipe Villa-Arenas, Nils Feldhus, Joachim Meyer, Robert P. Spang, and Sebastian Möller. 2024. The Role of Explainability in Collaborative Human-AI Disinformation Detection. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (Rio de Janeiro, Brazil) (FAccT '24)*. Association for Computing Machinery, New York, NY, USA, 2157–2174. doi:10.1145/3630106.3659031
- [110] Hendrik Schuff, Alon Jacovi, Heike Adel, Yoav Goldberg, and Ngoc Thang Vu. 2022. Human Interpretation of Saliency-based Explanation Over Text. In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency (Seoul, Republic of Korea) (FAccT '22)*. Association for Computing Machinery, New York, NY, USA, 611–636. doi:10.1145/3531146.3533127
- [111] Alfred Schutz. 1967. *The phenomenology of the social world*. Northwestern university press.
- [112] Abigail Sellen and Eric Horvitz. 2024. The Rise of the AI Co-Pilot: Lessons for Design from Aviation and Beyond. *Commun. ACM* 67, 7 (July 2024), 18–23. doi:10.1145/3637865
- [113] Ruoxi Shang, K. J. Kevin Feng, and Chirag Shah. 2022. Why Am I Not Seeing It? Understanding Users' Needs for Counterfactual Explanations in Everyday Recommendations. In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency (Seoul, Republic of Korea) (FAccT '22)*. Association for Computing Machinery, New York, NY, USA, 1330–1340. doi:10.1145/3531146.3533189
- [114] Yonadav Shavit, Sandhini Agarwal, Miles Brundage, Steven Adler, Cullen O'Keefe, Rosie Campbell, Teddy Lee, Pamela Mishkin, Tyna Eloundou, Alan Hickey, Katarina Slama, Lama Ahmad, Paul McMillan, Alex Beutel, Alexandre Passos, and David G. Robinson. 2023. *Practices for Governing Agentic AI Systems*. Technical Report. OpenAI. <https://cdn.openai.com/papers/practices-for-governing-agentic-ai-systems.pdf>
- [115] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. Hugginggpt: Solving AI tasks with ChatGPT and its friends in hugging face. *Advances in Neural Information Processing Systems* 36 (2023), 38154–38180.
- [116] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2023), 8634–8652.
- [117] Ben Shneiderman. 2020. Human-Centered Artificial Intelligence: Reliable, Safe & Trustworthy. *International Journal of Human-Computer Interaction* 36, 6 (2020), 495–504. arXiv:<https://doi.org/10.1080/10447318.2020.1741118> doi:10.1080/10447318.2020.1741118
- [118] Ben Shneiderman. 2022. *Human-centered AI*. Oxford University Press.
- [119] Parshin Shojaei*, Iman Mirzadeh*, Keivan Alizadeh, Maxwell Horton, Samy Bengio, and Mehrdad Farajtabar. 2025. The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity. <https://ml-site.cdn->

- apple.com/papers/the-illusion-of-thinking.pdf
- [120] Pradyumna Shome, Sashreek Krishnan, and Sauvik Das. 2025. Why Johnny Can't Use Agents: Industry Aspirations vs. User Realities with AI Agent Software. arXiv:2509.14528 [cs.HC] <https://arxiv.org/abs/2509.14528>
 - [121] HA SIMON. 1972. Theories of Bounded Rationality. *Decision and Organization* 22 (1972), 161–176.
 - [122] H. A. Simon. 1956. Rational choice and the structure of the environment. *Psychological Review* 63, 2 (1956), 129–138. doi:10.1037/h0042769
 - [123] Hao Song, Yiming Shen, Wenxuan Luo, Leixin Guo, Ting Chen, Jiashui Wang, Beibei Li, Xiaosong Zhang, and Jiachi Chen. 2025. Beyond the Protocol: Unveiling Attack Vectors in the Model Context Protocol (MCP) Ecosystem. arXiv:2506.02040 [cs.CR] <https://arxiv.org/abs/2506.02040>
 - [124] Joseph Spracklen, Raveen Wijewickrama, A H. M. Nazmus Sakib, Anindya Maiti, and Murtuza Jadliwala. 2024. We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs. *CoRR* abs/2406.10279 (2024). <https://doi.org/10.48550/arXiv.2406.10279>
 - [125] Kaya Stechly, Karthik Valmeekam, and Subbarao Kambhampati. 2025. On the self-verification limitations of large language models on reasoning and planning tasks. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. <https://openreview.net/forum?id=4O0v4s3IzY>
 - [126] Stephanie B. Steinhardt and Steven J. Jackson. 2015. Anticipation Work: Cultivating Vision in Collective Practice. In *Proceedings of the 18th ACM Conference on Computer Supported Cooperative Work & Social Computing* (Vancouver, BC, Canada) (CSCW '15). Association for Computing Machinery, New York, NY, USA, 443–453. doi:10.1145/2675133.2675298
 - [127] Sarah Sterz, Kevin Baum, Sebastian Biewer, Holger Hermanns, Anne Lauber-Rönsberg, Philip Meinel, and Markus Langer. 2024. On the Quest for Effectiveness in Human Oversight: Interdisciplinary Perspectives. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency* (Rio de Janeiro, Brazil) (FAccT '24). Association for Computing Machinery, New York, NY, USA, 2495–2507. doi:10.1145/3630106.3659051
 - [128] Perplexity Team. 2025. How People Use AI Agents — perplexity.ai. <https://www.perplexity.ai/hub/blog/how-people-use-ai-agents>. [Online; accessed 12-01-2026].
 - [129] Yuchen Tian, Weixiang Yan, Qian Yang, Xuandong Zhao, Qian Chen, Wen Wang, Ziyang Luo, Lei Ma, and Dawn Song. 2025. CodeHalu: Investigating Code Hallucinations in LLMs via Execution-based Verification. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 24 (Apr. 2025), 25300–25308. doi:10.1609/aaai.v39i24.34717
 - [130] Adam Tornhill, Markus Borg, Nadim Hagatulah, and Emma Söderberg. 2025. ACE: automated technical debt remediation with validated large language model refactorings. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 1318–1324.
 - [131] Sarah J. Tracy and Angela N. Gist-Mackey. 2024. Chapter 5 Qualitative Methods in Organizational Communication. In *Organizational Communication Theory and Research*, Vernon D. Miller and Marshall Scott Poole (Eds.). De Gruyter Mouton, Berlin, Boston, 85–104. doi:10.1515/9783110718508-006
 - [132] Andreas Tsamados, Luciano Floridi, and Mariarosaria Taddeo. 2025. Human control of AI systems: from supervision to teaming. *AI and Ethics* 5, 2 (01 Apr 2025), 1535–1548. doi:10.1007/s43681-024-00489-4
 - [133] Violet Turri, Katelyn Morrison, Katherine-Marie Robinson, Collin Abidi, Adam Perer, Jodi Forlizzi, and Rachel Dzombak. 2024. Transparency in the Wild: Navigating Transparency in a Deployed AI System to Broaden Need-Finding Approaches. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency* (Rio de Janeiro, Brazil) (FAccT '24). Association for Computing Machinery, New York, NY, USA, 1494–1514. doi:10.1145/3630106.3658985
 - [134] Amos Tversky and Daniel Kahneman. 1974. Judgment under Uncertainty: Heuristics and Biases. *Science* 185, 4157 (1974), 1124–1131. doi:10.1126/science.185.4157.1124
 - [135] Ruben S. Verhagen, Mark A. Neerincx, and Myrthe L. Tielman. 2025. Agent Allocation of Moral Decisions in Human-Agent Teams: Raise Human Involvement and Explain Potential Consequences. In *Proceedings of the 2025 ACM Conference on Fairness, Accountability, and Transparency* (FAccT '25). Association for Computing Machinery, New York, NY, USA, 2302–2317. doi:10.1145/3715275.3732157
 - [136] Ruotong Wang, Ruijia Cheng, Denae Ford, and Thomas Zimmermann. 2024. Investigating and Designing for Trust in AI-powered Code Generation Tools. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency* (Rio de Janeiro, Brazil) (FAccT '24). Association for Computing Machinery, New York, NY, USA, 1475–1493. doi:10.1145/3630106.3658984
 - [137] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable code actions elicit better LLM agents. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) (ICML '24). JMLR.org, Article 2054, 25 pages.
 - [138] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. 2024. OpenDevin: An Open Platform for AI Software Developers as Generalist Agents. *CoRR* abs/2407.16741 (2024). <https://doi.org/10.48550/arXiv.2407.16741>
 - [139] You Wang, Michael Pradel, and Zhongxin Liu. 2025. Are “Solved Issues” in SWE-bench Really Solved Correctly? An Empirical Study. *arXiv e-prints*, Article arXiv:2503.15223 (March 2025), arXiv:2503.15223 pages. arXiv:2503.15223 [cs.SE] doi:10.48550/arXiv.2503.15223

- [140] Yanlin Wang, Wanjun Zhong, Yanxian Huang, Ensheng Shi, Min Yang, Jiachi Chen, Hui Li, Yuchi Ma, Qianxiang Wang, and Zibin Zheng. 2025. Agents in software engineering: survey, landscape, and vision. *Automated Software Engg.* 32, 2 (Aug. 2025), 36 pages. doi:10.1007/s10515-025-00544-2
- [141] Miku Watanabe, Hao Li, Yutaro Kashiwa, Brittany Reid, Hajimu Iida, and Ahmed E. Hassan. 2025. On the Use of Agentic Coding: An Empirical Study of Pull Requests on GitHub. arXiv:2509.14745 [[cs.SE](http://cs.se/)] <https://arxiv.org/abs/2509.14745>
- [142] Laura Weidinger, Jonathan Uesato, Maribeth Rauh, Conor Griffin, Po-Sen Huang, John Mellor, Amelia Glaese, Myra Cheng, Borja Balle, Atoosa Kasirzadeh, Courtney Biles, Sasha Brown, Zac Kenton, Will Hawkins, Tom Stepleton, Abeba Birhane, Lisa Anne Hendricks, Laura Rimell, William Isaac, Julia Haas, Sean Legassick, Geoffrey Irving, and Iason Gabriel. 2022. Taxonomy of Risks posed by Language Models. In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency* (Seoul, Republic of Korea) (FAccT '22). Association for Computing Machinery, New York, NY, USA, 214–229. doi:10.1145/3531146.3533088
- [143] Ben Weidmann, Yixian Xu, and David J Deming. 2025. *Measuring Human Leadership Skills with AI Agents*. Working Paper 33662. National Bureau of Economic Research. doi:10.3386/w33662
- [144] Minghao Wu, Jiahao Xu, Yulin Yuan, Gholamreza Haffari, Longyue Wan, Weihua Luo, and Kaifu Zhang. 2025. (Perhaps) Beyond Human Translation: Harnessing Multi-Agent Collaboration for Translating Ultra-Long Literary Texts. *Transactions of the Association for Computational Linguistics* 13 (2025), 901–922.
- [145] Xin Xia, Lingfeng Bao, David Lo, Zhenchang Xing, Ahmed E. Hassan, and Shanping Li. 2018. Measuring program comprehension: a large-scale field study with professionals. In *Proceedings of the 40th International Conference on Software Engineering* (Gothenburg, Sweden) (ICSE '18). Association for Computing Machinery, New York, NY, USA, 584. doi:10.1145/3180155.3182538
- [146] Liwenhan Xie, Chengbo Zheng, Haijun Xia, Huamin Qu, and Chen Zhu-Tian. 2024. WaitGPT: Monitoring and Steering Conversational LLM Agent in Data Analysis with On-the-Fly Code Visualization. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology* (Pittsburgh, PA, USA) (UIST '24). Association for Computing Machinery, New York, NY, USA, Article 119, 14 pages. doi:10.1145/3654777.3676374
- [147] Yisen Xu. 2025. MUARF: Leveraging Multi-Agent Workflows for Automated Code Refactoring. In *2025 IEEE/ACM 47th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. 226–227. doi:10.1109/ICSE-Companion66252.2025.00071
- [148] Kaiwen Yan, Hongcheng Guo, Xuanqing Shi, Shaosheng Cao, Donglin Di, and Zhoujun Li. 2025. CodeLF: Benchmarking the Instruction-Following Capabilities of Large Language Models for Code Generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, Georg Rehm and Yunyao Li (Eds.). Association for Computational Linguistics, Vienna, Austria, 1272–1286. doi:10.18653/v1/2025.acl-industry.89
- [149] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: agent-computer interfaces enable automated software engineering. In *Proceedings of the 38th International Conference on Neural Information Processing Systems* (Vancouver, BC, Canada) (NIPS '24). Curran Associates Inc., Red Hook, NY, USA, Article 1601, 125 pages. doi:10.52202/079017-1601
- [150] Zhou Yang, Zhipeng Zhao, Chenyu Wang, Jieke Shi, Dongsun Kim, Donggyun Han, and David Lo. 2024. Unveiling Memorization in Code Models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) (ICSE '24). Association for Computing Machinery, New York, NY, USA, Article 72, 13 pages. doi:10.1145/3597503.3639074
- [151] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations*. https://openreview.net/forum?id=WE_vluYUL-X
- [152] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. 2024. CodeAgent: Enhancing Code Generation with Tool-Integrated Agent Systems for Real-World Repo-level Coding Challenges. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 13643–13658. doi:10.18653/v1/2024.acl-long.737
- [153] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. 2024. AutoCodeRover: Autonomous Program Improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) (ISSTA 2024). Association for Computing Machinery, New York, NY, USA, 1592–1604. doi:10.1145/3650212.3680384
- [154] Tianyu Zheng, Ge Zhang, Tianhao Shen, Xueling Liu, Bill Yuchen Lin, Jie Fu, Wenhui Chen, and Xiang Yue. 2024. OpenCodeInterpreter: Integrating Code Generation with Execution and Refinement. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 12834–12859. doi:10.18653/v1/2024.findings-acl.762
- [155] Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, Shayne Longpre, Kevin Meng, Rebecca Weiss, Fazl Barez, Fazl Gupta, Jwala Dhamala, Jacob Merizian, Mario Giulianelli, Harry Coppock, Cozmin Ududec, Jasjeet Sekhon, Jacob Steinhardt, Antony Kellermann, Sarah Schwettmann, Matei Zaharia, Ion Stoica, Percy Liang, and Daniel Kang. 2025. Establishing Best Practices for Building Rigorous Agentic Benchmarks. arXiv:2507.02825 [cs.AI] <https://arxiv.org/abs/2507.02825>
- [156] Mingchen Zhuge, Changsheng Zhao, Dylan R Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, et al. 2025. Agent-as-a-Judge: Evaluate Agents with Agents. In *Forty-second International*

- Conference on Machine Learning*.
- [157] Terry Yue Zhuo, Vu Minh Chien, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen GONG, James Hoang, Armel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, Naman Jain, Alex Gu, Zhoujun Cheng, Jiawei Liu, Qian Liu, Zijian Wang, David Lo, Binyuan Hui, Niklas Muennighoff, Daniel Fried, Xiaoning Du, Harm de Vries, and Leandro Von Werra. 2025. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=YrycTjllL0>
- [158] Theresa Züger, Philipp Mahlow, Daniel Pothmann, Katharina Mosene, Fabian Burmeister, Matthias Kettemann, and Wolfgang Schulz. 2025. Crediting Humans: A Systematic Assessment of Influencing Factors for Human-in-the-Loop Figurations in Consumer Credit Lending Decisions. In *Proceedings of the 2025 ACM Conference on Fairness, Accountability, and Transparency (FAccT '25)*. Association for Computing Machinery, New York, NY, USA, 1281–1292. doi:10.1145/3715275.3732086

A Appendix: Participant demographics

We provide a detailed table below that captures all participants' demographics, including their self-reported programming experience, software agent experience, usage.

No.	Gender	Country	Role	Programming experience	Software agent experience	Software agent usage frequency	Software agents used
P01	F	US	Software engineer / developer	>10 years	Experienced	Once a week	GitHub Copilot
P02	M	US	Software engineer/ developer	7-10 years	Very experienced	Daily	Cline; Cursor; GitHub Copilot
P03	M	US	Engineering manager	>10 years	Experienced	Daily	Claude Code; Cline; Cursor; GitHub Copilot
P04	M	India	Software engineer/ developer	1-3 years	Very experienced	Daily	Cursor; GitHub Copilot; OpenAI Codex
P05	M	US	Technical Program Manager	1-3 years	Experienced	Several times a week	Amazon Kiro; Cursor; GitHub Copilot
P06	M	US	Data science/ ML engineering	4-6 years	Somewhat experienced	Several times a week	Cursor
P07	M	US	Engineering manager	>10 years	Experienced	Daily	Cline; GitHub Copilot; Rubber Ducky
P08	M	US	Software engineer/ developer	>10 years	Experienced	Daily	GitHub Copilot
P09	M	Brazil	Software engineer/ developer	7-10 years	Very experienced	Daily	Amazon CodeWhisperer; GitHub Copilot; OpenAI Codex
P10	M	US	Software engineer/ developer	4-6 years	Experienced	Several times a week	Gemini; GitHub Copilot
P11	M	US	Software engineer/ developer	>10 years	Somewhat experienced	Daily	Amazon CodeWhisperer; Cursor; GitHub Copilot
P12	M	US	Software engineer/ developer	>10 years	Somewhat experienced	Several times a week	Gemini
P13	M	US	Software engineer/ developer	>10 years	Experienced	Daily	Cursor; GitHub Copilot
P14	F	US	Software engineer/ developer	>10 years	Experienced	Daily	Aider; Claude Code; GitHub Copilot
P15	M	US	Software engineer/ developer	>10 years	Experienced	Daily	Claude Code, GitHub Copilot; VsCode;
P16	M	US	Software engineer/ developer	>10 years	Very experienced	Daily	Codex CLI; Cursor; GitHub Copilot
P17	M	US	Software engineer/ developer	>10 years	Somewhat experienced	Daily	Claude Code; Cursor; GitHub Copilot

Table 1. Participant details (self-reported): Demographics, programming experience, software agent experience, software agent usage frequency, and software agents used.

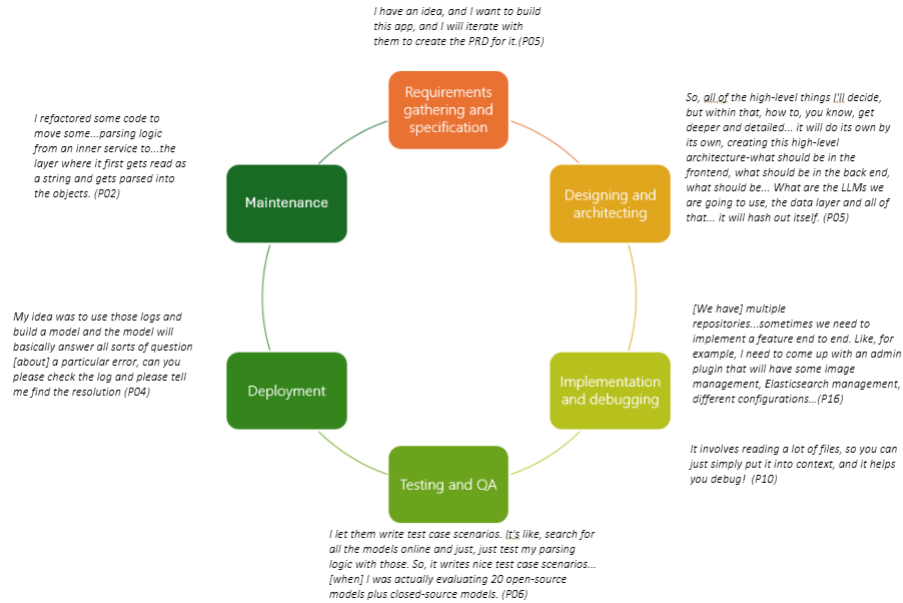


Fig. 1. Participants' description of scenarios in which they use AI agents for software development

B Appendix: Agent usage scenarios

Our study did not aim to quantify software agent usage patterns, but participants shared their real-world situations to illustrate and ground their oversight practices. We illustrate key examples in Figure 1. Participants' accounts revealed that AI agents are integrated across several core phases of developers' software development and engineering practices, automating various tasks such as drafting and validating product requirements documents, implementing end-to-end features, debugging code, analyzing storage logs, and code refactoring. Participants used agents across codebases implemented in different programming languages, including Python, Java, Go, and C#. These descriptions complement industry reports [4, 128] demonstrating agents pervasive use in automating a variety of programming tasks.

Software development is not limited to writing code or implementing features; it also involves continuously understanding and learning code [34, 145] and collaborating with other developers [71]. Our participants also used agents to support these activities. For instance, P16 used agents to catch-up on codebase changes after returning from paternity leave, while P06 leveraged agents to generate visualizations to communicate trends in their research findings.

One interesting insight from our data is that when working with production-grade enterprise software, participants usually delegated only small, well-bounded tasks to software agents—tasks that an agent can complete in a short execution window typically within a “few dozen seconds to a minute” (P12). This has implications for developers' oversight practices. In a few cases, agent run time was relatively longer: for proof-of-concept or prototype work (e.g., 20-25 minutes (P10)) and when participants were advanced power users (e.g., 15-20 minutes (P16)) who engaged agents in more extended tasks.

C Appendix: Interview Topic Guide

C.0.1 Introduction and Welcome. Thank you for participating in this study. We are interviewing people to understand how they use and oversee agents. Your participation is voluntary, and your responses will be anonymous and confidential. Results will be reported only in aggregate and when referred, your quotes will be de-identified. With your permission, Marvin will be recording this meeting to ensure accuracy.

C.0.2 Agent usage.

- (1) To begin, could you describe the scenarios in which you use coding agents? e.g., Github Copilot (agent mode), Cursor, Claude Code, Windsurf, OpenAI Codex, Mistral's Devstral (Open source)

C.0.3 Before deployment.

- (1) How do you decide which agent(s) to choose for which task i.e. which factors help you decide the agent you work with?
 - Do you look at leaderboards, model cards, or any benchmarks e.g., SWE-bench when selecting models to use, their training cut off?
- (2) Are there any tradeoffs that you encounter when choosing agents?
- (3) How have you learned to tailor your agent for your software project?
 - e.g., Custom instructions – to include best practices for your applications stack?
 - e.g., Connections to things like databases with MCP? Which MCP server to use, the tools available from it?
- (4) What risks related to agentic automation do you consider during this initial phase?
- (5) What evals do you put in place? What goals do they serve?
 - agents' accuracy – consistency with initial requirements?
 - agents' biases in reasoning or execution
 - agents' time to execution
 - agents' adherence to intended scope
 - agent not going off-task
 - agent not leaking sensitive information
 - agent not producing copyright information

C.0.4 At run-time.

- (1) What do you do when the agents are deployed?
- (2) What risks related to agentic automation do you consider when the agent is executing on a task?
- (3) How do you govern your agent(s) when they are deployed?
- (4) How do you intervene agents? Could you describe the types of interventions you make?

C.0.5 Post deployment.

- (1) How do you test the correctness of the agents' actions and outputs?
- (2) How do you define goodness for agents' actions and outputs?
- (3) Which artifacts do you inspect to determine this goodness?
- (4) What verification tools are available to you? e.g., manual, automated techniques, agents-as-judge, confidence scores, agents' justifications, etc.

C.0.6 Agent impact on workflow.

- (1) Do you ever feel stuck when using agents? What things do you do to get unstuck? Are you own our own, with your rubber duck, or with another team member when you troubleshoot?
- (2) How has your workflow changed, now that you have access to coding agents?
 - What activities do you do now that you didn't do earlier?
 - What activities do you not do when you are working with agents that you would otherwise do?

C.0.7 Concluding remarks. I'd like to open it up for questions that you may have. You will receive the \$40 giftcard in the email you provided earlier. Thank you for your time.

D Appendix: Four forms of oversight work

Table 2 captures the four forms of emergent oversight work and their descriptions as revealed in our exploratory analysis.

Oversight Work	Description
A priori control	A preventative form of oversight work that involves giving instructions to agents to direct and limit their workings <i>before</i> delegating tasks to them. A priori control includes configuring agent settings, constraining agent boundaries, and supplying global context.
Co-planning work	A proactive form of oversight work that involves pre-execution goal-setting activities that agents and developers jointly perform to establish common ground. Examples include drafting plans with agents, iterative prompting, and seeding partial solutions to agents.
Real-time monitoring	A reactive form of oversight work that involves observing agents and intervening, when needed, as they perform tasks. Examples includes observing agent reasoning and pausing/stopping agents mid-execution.
Post-hoc review	An evaluative form of oversight work that involves reviewing and correcting agent outputs after execution. Examples include verifying outputs, making adjustments when agents falter, and re-prompting to fix mistakes.

Table 2. The four forms of emergent agent oversight work

Received 13 January 2026